UNIVERSITY OF LEICESTER

# School of Computing and Mathematical Sciences

# CO7201 Individual Project

## Final Report

## Stock Control System

**Abhishek Jayeshbhai Patel**
**ajp124@student.le.ac.uk**
**259041375**

**Project Supervisor: Muhammad, Sadiq**
**Principal Marker: Vladimirova, Tanya**

**Word Count: 20316**
**04/09/2026**

## Declaration

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stage, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do these amounts to plagiarism and will be considered grounds for failure in this module and the degree examination.

Name: Abhishek Jayeshbhai Patel

Date: 04/09/2026

## Abstract

Stock Control System is a web-based stock control system for small and medium-sized organisations that have outgrown spreadsheet-based record keeping but cannot justify the cost or configuration effort of an enterprise resource planning package. The system supports the complete stock lifecycle — products, suppliers, warehouses, receiving, issuing, adjustment, transfer, batch and expiry tracking, procurement, stock-taking and reporting — and adds three intelligent capabilities on top of that transactional foundation: demand forecasting from historical movement data, replenishment recommendations that combine forecast demand with supplier lead times, and predictive alerts raised before a minimum threshold is breached.

The system is implemented as a Django and Django REST Framework application over PostgreSQL, with asynchronous processing on Celery and Redis, and a Next.js client. It comprises 44 database models across twelve domain applications, more than 40 view classes and over 50 custom endpoint actions, and 41 client routes gated by a 21-module by 7-action permission catalogue (147 possible module–action pairs).

The forecasting service does not commit to a single model. For each product it fits Holt exponential smoothing and an ARIMA (1,1,0) model, compares both against a simple moving average and a naive last-period baseline on a held-out week, and retains whichever achieves the lowest mean absolute error. Evaluated across four synthetic demand profiles designed to reproduce shapes observed in real inventory data, this competitive selection reduced mean absolute error against the naive baseline by 42.3%, 79.3% and 73.2% on trending, seasonal and stable-but-noisy series respectively. On an intermittent-demand profile it correctly selected the naive baseline itself and achieved no improvement — a result consistent with the published literature on intermittent demand and reported here as a genuine limitation rather than omitted. Replenishment recommendations matched manual recalculation in every test case, and predictive alerts fired in every scenario in which a stockout fell within the supplier's lead time while remaining silent where it did not.

The evaluation additionally documents fourteen defects identified in the delivered system during testing and code review, including a boundary error in the ABC classification routine and a set of unauthenticated development endpoints, on the view that a report which found nothing wrong would be less credible rather than more. These, and the design decisions that were reversed during development, are recorded in full in the critical reflection.

## Acknowledgements

I would like to thank my project supervisor, Muhammad Sadiq, for his guidance throughout this project, and for the insistence that the intelligent features be evaluated properly rather than merely demonstrated to produce output. I am also grateful to my principal marker, Tanya Vladimirova, for her feedback at the preliminary report stage, and to those who agreed to work through an unfinished system and report honestly where it confused them.

# Table of Contents

# 1. Introduction

## 1.1 Motivation

Almost every organisation that keeps physical goods must answer the same three questions repeatedly: what do we have, where is it, and when do we need more. Large companies answer these questions with enterprise resource planning software that costs a great deal and takes months to configure. Very small operations answer them by looking on the shelf. The awkward middle ground — a pharmacy with two branches, a school's technician store, a restaurant group, a small manufacturing workshop — tends to answer them with a spreadsheet that one person maintains and everybody else distrusts.

The problem is not that a spreadsheet cannot hold inventory data; it can. The problem is that a spreadsheet records a state rather than a history. When a quantity cell changes from 45 to 60, the information that twelve units were sold and three were damaged is destroyed. Without that history there is nothing to reason about: no forecast, no reorder calculation, no audit trail, no way to notice that one member of staff writes off unusually large quantities on Friday afternoons. This observation shaped the whole design of the project: Stock Control System is built around an append-only ledger of stock movements rather than an editable quantity field, and everything else — reporting, forecasting, alerting, anomaly detection — is derived from that ledger (Section 5.6).

## 1.2 Problem Statement

Existing stock control software sits at two extremes. At one end are full enterprise resource planning suites such as SAP Business One and Odoo — capable and well-tested, but assuming a dedicated implementation partner, a configuration phase measured in months, and a licence budget beyond the reach of a small organisation. At the other end are lightweight inventory apps and point-of-sale add-ons that record transactions competently but stop at description: they report the current quantity and produce a valuation, but they do not indicate what is likely to happen next week.

The gap this project set out to fill is a system that is small enough to be set up by a non-specialist in an afternoon, that records stock movements in a way that makes the history recoverable and auditable, and that uses that history to give forward-looking advice rather than only backward-looking reports. The third point is what makes this an intelligence project rather than a routine data-management exercise: moving from a descriptive system to a decision-support system means answering questions such as how much stock is likely to be needed in the next thirty days, which items should be reordered first, and which product will run out before the next delivery arrives. Those questions require forecasting, and forecasting on the kind of data a small business has — short series, gaps, sudden spikes — is a genuinely difficult problem, not a solved one.

## 1.3 Aim

The aim of this project is to design, build and evaluate an intelligent stock control platform that delivers accurate transactional inventory management alongside demand forecasting, explainable

replenishment suggestions and predictive alerting, so that a small or medium-sized organisation can plan its purchasing on evidence rather than intuition.

## 1.4 Objectives

The aim decomposes into six objectives, carried forward from the preliminary report:

1. **Analyse the requirements** of a practical stock control system through review of the inventory management literature and of existing commercial systems.
2. **Design the core modules** for products, suppliers, receiving, issuing, adjustment, transfer, alerting and reporting, using a data model that preserves movement history.
3. **Implement authentication and role-based access control** using JWT tokens across administrator, manager and staff roles.
4. **Maintain data integrity** through validation, database constraints, atomic transactions and an immutable audit trail.
5. **Implement and evaluate three intelligent features**: demand forecasting, replenishment recommendation and predictive low-stock alerting.
6. **Assess the system** through functional, integrity, security, performance and usability testing.

## 1.5 Scope of the Delivered System

It is worth setting the scope out early, because the delivered system is considerably larger than the essential requirement list. A user authenticates and receives a JWT access token; what they see afterwards depends on their resolved permissions (Section 5.8), so a staff member who can record stock movements never sees the audit log or user administration screens at all.

From the dashboard a manager may maintain products, categories and suppliers, define storage locations, receive goods against a batch number and expiry date, issue stock, correct a discrepancy with a mandatory written justification, transfer stock between locations, conduct a physical stock-take, raise a purchase order, record a supplier return and issue an invoice. Every such operation writes an audit entry carrying before-and-after state.

On top of that transactional core, the system generates a demand forecast per product, ranks products by reorder priority with a suggested quantity and a structured explanation, raises predictive stockout alerts, flags statistically unusual movements using an Isolation Forest, classifies products into ABC value tiers overnight, sends expiry warnings at thirty, fifteen and seven days, and answers natural-language inventory questions through a read-only chat assistant.

## 1.6 Principal Technical Challenges

Five aspects proved more demanding than anticipated at the planning stage.

- **Keeping quantities correct**. A stock issue is not one write. It decrements a balance row, consumes one or more batches in expiry order, consumes one or more cost layers to compute cost of goods sold, writes a transaction row, writes an audit entry, and triggers anomaly scoring and alert re-evaluation. If any step fails partway through, the recorded quantity and the recorded history disagree, and once they disagree there is no reliable way to reconcile

them. Getting the transaction boundaries and row locking right (Section 5.6, Section 6.2) took several iterations.

- **Forecasting on thin data.** Textbook forecasting assumes a long, regular series. A small-business product might have forty days of history with a dozen zero-demand days in it, on which a single fixed model can produce something confidently wrong. This is addressed by declining to commit to any single model (Section 6.3).
- **Designing permissions** that are fine-grained without becoming unusable. Three fixed roles are simple to implement but too rigid for real organisations, while an unconstrained permission matrix is flexible but overwhelming. The resolution adopted is described in Section 5.8.
- Scope discipline, which was not consistently maintained. Chapter 9 examines two features that, on reflection, should not have been undertaken ahead of testing.

## 1.7 Risks and How They Were Managed

The preliminary report identified four project risks. Recording what occurred against each is more informative than restating the original plan, so the outcome column below is written with hindsight.

| Risk | Mitigation Planned | What Occurred in Practice |
|---|---|---|
| Forecasting models perform poorly on sparse data. | Start with exponential smoothing; fall back to average-demand rules if accuracy does not beat the naive baseline, and document it. | Materialised. A single model was unreliable across products, so the design changed to competitive selection with the naive baseline itself as a candidate (Section 6.3). Section 8.2 reports where it still fails. |
| Stock quantity errors from concurrent transactions. | Database transactions and optimistic locking; repeated test coverage on all stock-modifying endpoints. | Mitigated, by pessimistic row locking (select_for_update) rather than optimistic locking (Section 5.6, Section 6.2) — correct by design, and since verified under real concurrent load with zero errors and zero lost updates (N02, Section 8.6). |
| Database corruption or migration failure. | Daily pg_dump backups; migrations tested on staging before being applied. | Did not materialise in production terms, but a related defect appeared: the migration history cannot be replayed cleanly on SQLite (D10, Section 9.3). |
| Security vulnerability in authentication or data handling. | JWT authentication, input validation, ORM-only queries, CORS configuration and a dedicated security testing session. | Partly materialised. The authentication path is sound and was tested (Section 7.4), but three unauthenticated debug endpoints survived into the delivered code (D2, Section 9.3). |

Table 1.1 — Project Risks from the Preliminary Report and Their Actual Outcomes

Two of the four risks materialised in some form. The one not anticipated at all is the one that caused the most rework: that adding features would crowd out testing, discussed in Chapter 9 as the central process failure of the project.

## 1.8 Structure of This Report

Chapter 2 reviews the inventory theory and forecasting literature the project rests on and compares four commercial systems. Chapter 3 states the requirements and records how they changed, and Chapter 4 the development methodology. Chapter 5 covers system design — architecture, technical specification, use cases, data model, permissions and the intelligence layer. Chapter 6 is the implementation chapter and contains the highlighted areas of source code, with screenshots. Chapter 7 covers testing, Chapter 8 evaluation against the requirements, Chapter 9 critical reflection including the defects found, and Chapter 10 conclusions and future work.

# 2. Background and Related Work

## 2.1 The Inventory Problem

Inventory management is among the oldest quantitative problems in operations research, and the central tension has not altered since it was first formalised by Harris [1]: holding stock incurs cost in capital, space, insurance and spoilage, while failing to hold it incurs cost in lost sales, idle production and emergency purchasing. What has changed is who makes that trade-off — a planning function in a large organisation, and whoever happens to notice the empty shelf in a smaller one. Silver, Pyke and Thomas [2] frame inventory management as a problem of policy rather than record-keeping, an observation that shaped the whole project: a system that only records does not address the underlying problem, which is one of decision.

## 2.2 Classical Inventory Theory

Three classical results shaped the design of the intelligence layer directly.

### ABC classification

ABC analysis [2] sorts of stock items by annual consumption value and divides them into three tiers. Class A items are the small proportion of products — often around 20% — that account for most of the value, typically 70–80%. Class B is the middle band and Class C the long tail of items that individually contribute very little. The practical use is attention allocation: it is not economically sensible to apply tight control to every product, so tight control is applied to the A items and simple rules to the C items. Stock Control System implements this as an overnight Celery task, analytics.tasks.classify_products_abc, which computes annual consumption value per product from the movement ledger, sorts descending, accumulates the percentage share and assigns a class. The boundary condition in this implementation proves to be slightly wrong; this was discovered during evaluation and is discussed in Section 8.5 and Section 9.3.

### The reorder point model

The reorder point is the stock level at which a replenishment order should be placed. In its standard form, ROP = (average daily demand × supplier lead time in days) + safety stock. The first term covers expected consumption while waiting for delivery; the second is a buffer against demand being higher than average or the supplier being later than promised. Silver, Pyke and Thomas [2] set safety stock from the standard deviation of demand during lead time and a target service level; Stock Control System uses a simpler rule of two days of average demand instead, for reasons explained in Section 6.4 — in short, the statistically correct formula needs a reliable variance estimate that a forty-day, sparse series cannot support.

### Economic order quantity

Once the reorder point establishes when to order, the economic order quantity (EOQ) model addresses how much: EOQ = √((2 × annual demand × ordering cost) / annual holding cost per unit), balancing the fixed cost of placing an order against the cost of holding the resulting stock. The model's

assumptions — constant demand, instant replenishment, no quantity discounts — rarely hold exactly, but it gives a sensible order of magnitude and prevents the system from recommending trivially small order quantities. Stock Control System uses a fixed ordering cost of £50 and estimates annual holding cost as 15% of unit price; both are business parameters that should be configurable per organisation but are currently hard-coded, a limitation recorded as D12 in Section 9.3.

## 2.3 Demand Forecasting

Hyndman and Athanasopoulos [3] are the standard modern reference for the two families of method relevant here: exponential smoothing and ARIMA. **Exponential smoothing** produces a forecast as a weighted average of past observations with weights that decay geometrically as observations age; Holt's linear method, used in Stock Control System, adds a trend component so the forecast can follow a series that is steadily rising or falling. Holt was chosen over Holt-Winters because seasonal estimation needs at least two full seasonal cycles of data, which cannot be assumed to exist for a given product. **ARIMA** models a series in terms of its own past values, the differences between consecutive values, and past forecast errors; Stock Control System fits ARIMA (1,1,0), deliberately conservative, since fully automatic order selection is slow and can fail badly on short series — a model that is mediocre reliably is preferable to one that is excellent most of the time and catastrophic occasionally.

Carbonneau, Laframboise and Vahidov [4] compared machine-learning methods, including neural networks and support vector machines, against traditional statistical forecasting on supply-chain demand data, and found that machine-learning methods can outperform statistical ones on non-linear demand patterns, but the margin was smaller than the literature enthusiasm suggests and the methods needed substantially more data than is available per product here — one reason a neural forecasting model was not attempted; a recurrent network trained on ninety observations would be an exercise in overfitting.

### The intermittent demand problem

Syntetos, Boylan and Disney [5] surveyed fifty years of inventory forecasting research and made a point that proved directly relevant to the results in Section 8.2: when demand is intermittent — many periods with zero demand and occasional bursts — standard forecasting methods perform badly, and the choice of method has a measurable effect on service levels and holding costs. Croston's method and its variants were developed specifically for this case. Croston's method was not implemented in the delivered system, and the evaluation in Section 8.2 shows exactly the failure Syntetos et al. [5] predict: on an intermittent series the system falls back to the naive baseline and achieves no improvement at all. That result is reported rather than dropped from the evaluation, because an honest, predictable failure is more useful than a table of uniformly good numbers.

## 2.4 Anomaly Detection

Liu, Ting and Zhou [6] introduced Isolation Forest, which departs from the density- and distance-based outlier methods that preceded it: rather than modelling what normal data looks like and measuring deviation from it, Isolation Forest builds random binary trees that split the feature space at random points and measures how many splits are needed to isolate each point. Outliers, being few and

different, are isolated in fewer splits. The method has linear time complexity, low memory requirements, and — critically for this project — needs no labelled examples of fraud, which do not exist and never will for this dataset. Stock Control System scores every stock issue and adjustment on five features — quantity, hour of day, day of week, acting user, and length of the stated reason — and flags outliers for review (Section 6.6).

## 2.5 Software Engineering Foundations

Two general software engineering ideas shaped the design more than any inventory-specific source. The first is event sourcing: store the sequence of changes and derive the current state. Sommerville [7] and Pressman and Maxim [8] both favours append-only records where auditability is required and applied to inventory this means a movement is an immutable fact while quantity on hand is a derived aggregate (Section 5.6). The second is that an audit log must be structurally immutable rather than immutable by convention — the model itself must refuse updates, which is how Activity Log is built (Section 6.7).

## 2.6 Review of Existing Systems

Four systems occupying adjacent segments of the same market were examined to establish how the problem is conventionally decomposed and where the gap lies: Zoho Inventory [9], Odoo Inventory [10], SAP Business One [11] and Katana Cloud Inventory [12]. The comparison below is based on each vendor's published product documentation.

| System | Positioning | Forecasting | Explains Its Advice | Setup Effort | Cost |
|---|---|---|---|---|---|
| Zoho Inventory | SME, cloud, multichannel retail | Reorder points, manual thresholds | No | Low (hours) | Low–moderate subscription |
| Odoo Inventory | SME to mid-market, open source with paid apps | Statistical forecasting in paid modules | Partially | Moderate to high | Free core, paid apps and hosting |
| SAP Business One | Mid-market ERP | MRP with demand planning | Limited | High (months, partner-led) | High licence and implementation |
| Katana Cloud Inventory | Small manufacturing | Basic reorder alerts | No | Low–moderate | Moderate subscription |
| Stock Control System (this project) | SME, single deployment | Adaptive model selection per product | Yes — structured explanation on every recommendation and alert | Low | Self-hosted |

Table 2.1 — Comparison of Existing Inventory Systems Against Stock Control System

Three observations follow from this comparison. First, every one of these systems implements a reorder point, but almost all of them treat it as a number the user types in rather than a value the system derives — that is a threshold, not a forecast. Stock Control System reorder point is recalculated

from observed demand and supplier lead time each time recommendations are generated. Second, where forecasting does exist it is usually a black box: the user sees a predicted quantity but not the model, the error metrics or the inputs, which is a real adoption barrier — if a manager cannot see why the system suggested a given quantity, the manager will order what they always order. Third, the gap is not a single missing feature so much as a missing combination: forecasting, reorder calculation and predictive alerting each exist somewhere individually, but what does not exist at the small-business end of the market is a system that does all three, explains itself, and can be deployed without a consultant.

This comparison concerns design approach rather than production readiness. Stock Control System is a single-developer academic project evaluated on synthetic data over a period of months, whereas the systems above are mature products with established production use, support, tax compliance and integration ecosystems that this project does not attempt to replicate.

## 2.7 Technology Choices

**Django and Django REST Framework** [13][14] were chosen for the backend. Django's ORM handles relational modelling and schema migration and provides transaction management and row-level locking as first-class facilities; Django REST Framework contributes serialisation, view sets, filtering, pagination and an extensible permission hook. The alternative considered was Fast API, which is faster and more modern but would have required assembling the ORM, migrations, admin and authentication from separate libraries — for a project of this duration, the integrated framework was the lower-risk choice.

**PostgreSQL** [15] was chosen for the database (SQLite is used in local development). The deciding factor was ACID-compliant transactions with row-level locking: concurrent stock arithmetic is precisely the case in which weaker transactional guarantees produce incorrect results. PostgreSQL also supports JSON columns natively, used for the explanation payloads, permission overrides and audit before/after snapshots.

**Next.js with React** [16] was chosen for the frontend: the App Router's file-based routing maps cleanly onto a system of over forty screens, Tailwind CSS [17] handles styling without a separate stylesheet to keep in sync, and Radix primitives through the shadcn/ui pattern give dialogs, dropdowns and sheets accessible behaviour by default.

**Celery with Redis ,AI/ML** handles background work — forecast generation, ABC classification, expiry checking, email delivery and scheduled reports all need to run on a timer or off the request path, and Celery Beat with the database scheduler gives a small set of periodic tasks that survive restarts (Table 6.2).

One decision would be reversed on reflection: the frontend was written in JavaScript rather than TypeScript; on the assumption this would allow faster progress. That assumption held for the first few weeks; thereafter, time lost to mismatched component properties and API response fields renamed without a corresponding client update outweighed any saving (Section 9.5).

# 3. Requirements

## 3.1 How the Requirements Were Established

The requirements came from three sources: the background reading in Chapter 2, which established what a stock control system needs to be theoretically sound; the review of existing commercial systems, which showed how the problem is normally divided into modules; and direct prior experience of a small retail business relying on spreadsheet-based stock tracking, which is where the emphasis on accountability and non-technical usability originates.

The three-tier structure of Essential, Recommended and Optional requirements was adopted. This tiering proved consequential in practice: when the schedule slipped in week six (Section 4.2), the explicit ordering meant the decision as to what should be reduced in scope had already been taken in week one.

## 3.2 Essential Requirements

| ID | Requirement | Description |
|---|---|---|
| E01 | Authentication and RBAC | Email/password login issuing JWT tokens; Admin, Manager and Staff roles operating within defined permission sets; account lockout after repeated failures. |
| E02 | Product and category management | Full lifecycle for stock items including SKU, description, unit price, minimum and reorder levels, category assignment and active status. |
| E03 | Supplier management | Supplier records with contact details, lead time in days and status. |
| E04 | Stock in | Record receipt of goods against a product, supplier, location, quantity and unit cost. |
| E05 | Stock out | Record issue or sale of goods, decrementing the correct balance and rejecting over-issue. |
| E06 | Stock adjustment | Correct a discrepancy with a mandatory written reason, recording the before and after quantity. |
| E07 | Real-time dashboard | Overview of total inventory value, items below threshold, out-of-stock count and recent activity. |
| E08 | Demand forecasting | Analyse historical movement data to predict future demand per product. |
| E09 | Predictive low-stock alerts | Warn of an expected stockout before the minimum threshold is breached. |
| E10 | Smart reorder recommendations | Combine forecast demand, current stock and supplier lead time into a suggested order quantity. |
| E11 | Reporting suite with export | Inventory, movement, low-stock, forecast, purchase and sales reports with PDF and Excel export. |
| E12 | Search and filtering | Search across products, suppliers and transactions with filters on the main list views. |
| E13 | Immutable audit trail | Append-only log of every state-changing action with actor, timestamp and before/after values. |

| ID | Requirement | Description |
|---|---|---|
| E14 | Settings and configuration | Organisation-level settings for currency, default thresholds, valuation method, security policy and forecast parameters. |

Table 3.1 — Essential Requirements (E01–E14)

## 3.3 Recommended Requirements

| ID | Requirement | Status |
|---|---|---|
| R1 | Barcode and QR code generation per product | Delivered |
| R2 | Purchase order management with supplier and expected delivery date | Delivered |
| R3 | Stock transfers between locations with atomic quantity updates | Delivered |
| R4 | Batch and lot tracking from receipt to issue | Delivered |
| R5 | Expiry date tracking with FEFO enforcement and alerts at 30, 15 and 7 days | Delivered |
| R6 | Supplier return management with automatic stock adjustment | Delivered |
| R7 | Advanced reporting covering purchase, sales and stock valuation | Delivered |
| R8 | Physical stock-take scheduling module | Delivered |

Table 3.2 — Recommended Requirements and Delivery Status

## 3.4 Optional Requirements

| ID | Requirement | Status |
|---|---|---|
| O1 | Automated ABC classification by inventory value | Delivered, with a boundary defect — see Section 9.3, D1 |
| O2 | Anomaly detection using Isolation Forest | Delivered |
| O3 | AI chatbot assistant for natural-language stock queries | Delivered |
| O4 | Supplier performance scoring | Partially delivered — fields and scoring logic exist, but the automatic recalculation job was not written |

Table 3.3 — Optional Requirements and Delivery Status

Three further items appeared during development that were not in the original plan: customer records and invoicing, a transactional email system with templates and a delivery queue, and scheduled report delivery. These are recorded in Section 9.4 as scope added at a cost — the time they consumed is discussed candidly in Section 9.2.

## 3.5 Non-Functional Requirements

| ID | Requirement | Target |
|---|---|---|
| N01 | Stock arithmetic correctness | Balances must match the sum of movements in 100% of transaction test cases. |
| N02 | Concurrency safety | Concurrent movements against the same product and location must not lose updates. |
| N03 | Report responsiveness | Reports generate and export in under 5 seconds on the seeded dataset. |
| N04 | API response time | List endpoints return within 500 ms for a seeded organisation. |
| N05 | Access control | Unauthorised access blocked in 100% of negative test cases. |
| N06 | Audit integrity | Audit records cannot be modified or deleted through any application path. |
| N07 | Usability for non-technical staff | A user with no training completes the core stock-in and stock-out tasks without assistance. |
| N08 | Forecast accuracy | Mean absolute error lower than a naive last-period baseline on held-out data. |

Table 3.4 — Non-Functional Requirements and Targets

## 3.6 Requirements-to-Objectives Traceability

| Objective | Related Requirements |
|---|---|
| 1. Analyse requirements | Underpins all Essential, Recommended and Optional requirements |
| 2. Design core modules | E01–E07, E11–E14, R1–R8 |
| 3. JWT authentication and RBAC | E01 |
| 4. Data integrity | E04–E06, E13, N01, N02, N06 |
| 5. Intelligent features | E08–E10, O1–O4 |
| 6. Evaluation | N03–N05, N07, N08, and the success criteria in Table 3.6 |

Table 3.5 — Requirements-to-Objectives Traceability

A requirement-by-requirement traceability matrix, mapping each requirement to its implementing code and verifying test, is given in Appendix A; the version there covers a representative subset rather than every line item, with the full set of verdicts given in Table 8.1.

## 3.7 Changes Since the Preliminary Report

Four requirements changed materially between the preliminary report and delivery.

**Multi-tenancy was added**. The preliminary report described a single-organisation system. Early in design it became clear that scoping every query to an organisation from the start was almost free, whereas retrofitting it later would touch every query in the codebase, so an Organization model and a foreign key on every top-level entity were introduced (Section 5.7). This proved to be one of the

better decisions made in the project; it also simplified the permission model, since permissions naturally scope to an organisation.

**The role model was generalised**. The plan specified three fixed roles. The delivered system implements a permission catalogue of 21 modules and 7 actions (147 possible pairs), with named roles carrying preset grants over that catalogue and per-user overrides applied on top (Section 5.8). The three named roles remain the default for a new deployment, but an administrator may now define a new role without any code change. The cost was roughly a week of extra work and a noticeably more complex permission check.

**Forecasting was made competitive rather than fixed**. The plan was to begin with exponential smoothing and add ARIMA if time permitted. The delivered service fits both, together with a simple moving average and a naive baseline and selects whichever performs best on held-out data for the individual product (Section 6.3). The initial single-model implementation produced clearly erroneous forecasts for some products and satisfactory ones for others, with no reliable rule identified in advance for predicting which; deferring the choice to the data proved simpler than attempting to anticipate it.

**External context enrichment was added, and its value is not established**. Late in the project, services were added that adjust the forecast using weather data, public holiday calendars, local event listings and search-trend data. The premise is defensible — adverse weather does alter demand for certain product categories — but the multipliers are unvalidated heuristics whose effects compound multiplicatively, and this is discussed critically in Section 9.3 (D5).

## 3.8 Evaluation Criteria

Each requirement has an associated success criterion, so that "delivered" means something checkable rather than "code exists".

| Feature | Success Criterion |
|---|---|
| Login and role-based access | Unauthorised access blocked in 100% of negative test cases |
| Stock management | Inventory calculations match expected results in 100% of transaction test cases |
| Reporting | Reports generate and export in under 5 seconds for the seeded dataset |
| Demand forecasting | MAE lower than the naive last-period baseline on held-out test data |
| Smart reorder recommendations | Reorder points and quantities match manual recalculation for all test products |
| Predictive low-stock alerts | Alert generated before threshold breach in 100% of simulated declining-stock scenarios |
| Audit trail | Update and delete attempts rejected at model level in 100% of attempts |
| Anomaly detection | Injected outliers flagged; false positive rate on normal traffic reported and discussed |

Table 3.6 — Success Criteria Used to Judge Each Feature

# 4. Development Methodology

## 4.1 Selection of a Development Approach

The project ran for approximately twelve weeks with a single developer, a fixed deadline and a requirement set known in advance to be larger than could comfortably fit. That combination precludes a strict Waterfall process, which assumes stable requirements and a trustworthy estimate, and equally precludes textbook Scrum, whose ceremonies exist principally to coordinate a team.

The approach adopted was incremental delivery in fixed weekly increments. Each week had a nominated deliverable that was supposed to be working and demonstrable by the end of it, and each increment built on the last. The governing rule was that the main branch should always remain in a working state: a feature left incomplete at the end of a week was either finished or placed behind a conditional check, so the remainder of the system continued to operate. This corresponds to what Sommerville [7] terms incremental development, and it suited the project because it front-loads risk — the transactional ledger and the forecasting engine were scheduled early so any failure there would occur while there was still time to change direction.

## 4.2 Iteration Plan and Outcomes

| Week | Planned | Actual |
|---|---|---|
| 1 | Project setup, database design, architecture | Completed. Design took longer than planned; migrations were rewritten twice. |
| 2 | Authentication, users, roles, permissions | Completed, but the permission model grew from three fixed roles to the full catalogue, absorbing part of week 3. |
| 3 | Products, categories, suppliers, locations | Completed. |
| 4 | Stock in, stock out, adjustments | Completed. Hardest week; batch consumption and cost layering were rewritten three times. |
| 5 | Dashboard, reporting, audit trail | Completed. |
| 6 | AI and intelligent features | Started. Forecasting took substantially longer than one week and ran into week 7. |
| 7 | Testing and bug fixing | Partially done. Forecasting overran; testing was compressed. |
| 8 | Recommended tier — batches, expiry, transfers, POs | Completed. |
| 9 | Optional tier — ABC, anomaly detection, chatbot | Completed. |
| 10 | Evaluation and documentation | Evaluation harness written; several defects found. |
| 11–12 | Final report | Report writing, plus fixes for defects that were safe to fix without invalidating measurements. |

Table 4.1 — Planned Iteration Schedule Against What Occurred in Practice

The most instructive feature of this schedule is the overrun in weeks 6 and 7. One week had been allocated to the intelligent features, an estimate that in hindsight was unrealistic. Forecasting alone took eight or nine working days, most spent discovering that the first implementation produced implausible output on short series and then working out what to do about it. The cost was absorbed by compressing the testing week, and the consequence is visible in the test coverage discussed in Section 7.2.

## 4.3 Version Control and Working Practice

The project lives in a Git repository split into Code/Backend and Code/Frontend. Development proceeded directly on the main branch rather than on feature branches, which is defensible for a single developer with no concurrent contributor, but renders the history less clean than it might be. Commits were made upon completion of working functionality, with messages that describe the change rather than the intention behind it, which is the less useful of the two conventions.

## 4.4 Development Environment

Backend development used Python 3 with a virtual environment, PostgreSQL running locally, and Redis as the Celery broker. CELERY_TASK_ALWAYS_EAGER is set to True in local configuration, so background tasks execute synchronously and no separate worker process is required for most development. The frontend was developed against a current Node LTS release. Configuration is read from environment variables through Django-environ, with. env.example documenting every variable and no secret committed to the repository. API documentation is generated by drf-spectacular at /api/docs/, which proved more useful than expected — a live, always-correct endpoint list made frontend integration considerably faster.

## 4.5 Reflection on the Development Process

Two aspects of the process proved effective. Scheduling the higher-risk work early meant the stock ledger was sound by the end of week 4, and all subsequent work could depend on it; and the tiered requirement list meant that when the schedule slipped, the decision as to what should be omitted had already been taken in week 1. Two aspects proved ineffective: testing was treated as a scheduled phase rather than a continuous practice, and scope was permitted to expand beyond the agreed tiers. Both are examined in Chapter 9.

# 5. System Design

## 5.1 Architecture Overview

Stock Control System uses a three-tier architecture with an additional asynchronous processing tier. The tiers are separated by clear interfaces, so each can be reasoned about, tested and in principle scaled independently.

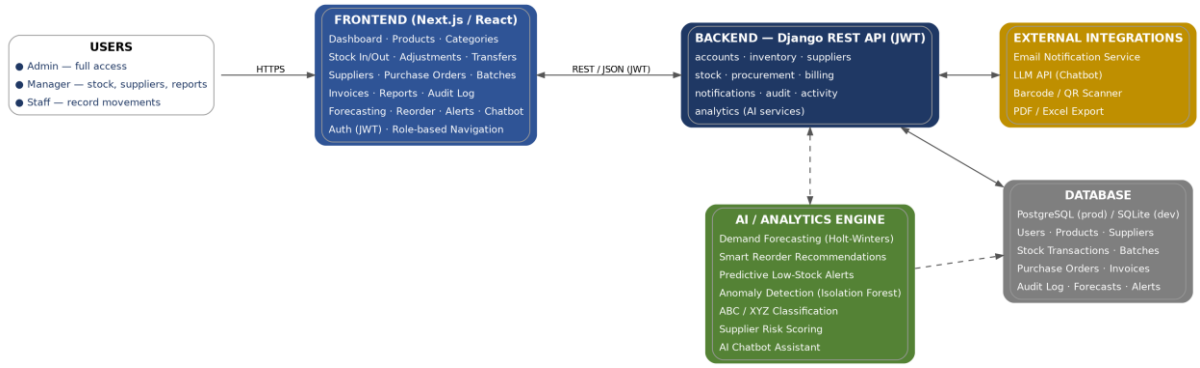


Figure 5.1 — System Architecture

The **presentation tier** is a Next.js application running on Node. It holds no business logic: it renders data, collects input, validates that input for user convenience, and calls the API. Authorisation decisions displayed in the interface — which menu items appear, which buttons are enabled — are convenience only; the same decisions are enforced independently on the server, because a client-side check is a usability feature and not a security control.

The **application tier** is the Django REST Framework API. It owns every business rule: what a valid stock movement looks like, how a reorder point is calculated, who may see an audit log. All state changes go through service classes rather than through views directly, keeping the rules in one place and testable without HTTP.

The **data tier is PostgreSQL**, relied upon for more than storage: foreign key constraints, unique constraints, check constraints on non-negative quantities, and, above all, transactional isolation with row-level locking.

The **asynchronous tier** is Celery with a Redis broker and the database-backed Beat scheduler, running the periodic tasks listed in Table 6.2.

## 5.2 Technical Specification

The principal server-side dependencies are Django, Django REST Framework and PostgreSQL; the analytical components are pandas [18], statsmodels [19] and scikit-learn [20]; the client is built on Next.js with Tailwind CSS. The system comprises 44 database models across twelve domain applications, more than 40 API view classes with over 50 additional custom endpoint actions for operations that are not plain CRUD, and 41 client routes.

| Layer | Technology | Purpose |
|---|---|---|
| Frontend framework | Next.js (App Router) | File-based routing, layouts and rendering across 41 screens |
| UI library | React | Component model for the interface |
| Styling | Tailwind CSS | Utility-first styling with no separate stylesheet to maintain |
| Components | Radix UI primitives via shadcn/ui | Accessible primitives — dialog, select, tabs, sheet, popover |
| Tables | TanStack React Table | Sorting, filtering and pagination on list views |
| Charts | Recharts | Forecast, movement and dashboard charts |
| Forms | react-hook-form + zod | Form state with schema validation shared across pages |
| Client state | Zustand | Auth store and UI state, persisted per "remember me" |
| Export | SheetJS (xlsx), react-to-print | Client-side Excel export and PDF printing |
| Backend framework | Django | ORM, migrations, admin, transactions, row-level locking |
| API layer | Django REST Framework | Serializers, viewsets, filtering, pagination, permissions |
| Authentication | djangorestframework-simplejwt | JWT access and refresh tokens with rotation and blacklisting |
| API documentation | drf-spectacular | OpenAPI 3 schema and Swagger UI at /api/docs/ |
| Database | PostgreSQL (psycopg2 driver) | ACID transactions and row-level locking for stock arithmetic |
| Task queue | Celery + Redis + django-celery-beat | Periodic tasks, email delivery, forecast generation |
| Statistical models | statsmodels | Holt exponential smoothing and ARIMA |
| Machine learning | scikit-learn, pandas | Isolation Forest for anomaly detection |
| Assistant | openai client (Groq or OpenAI) | Optional tool-calling loop; offline keyword mode by default |

Table 5.1 — Delivered Technology Stack and Purpose

Three declared dependencies — pyotp, Authlib and pgvector — are present in requirements.txt but are not exercised by the delivered feature set; they were added early, when multi-factor authentication, federated login and vector search remained on the optional list. Retaining unused dependencies is poor practice and their removal is noted as a housekeeping item in Section 9.3.

## 5.3 Actors and Use Cases

Four actors interact with the system. Three are human roles and one is the scheduler, which acts on the system with no user present.

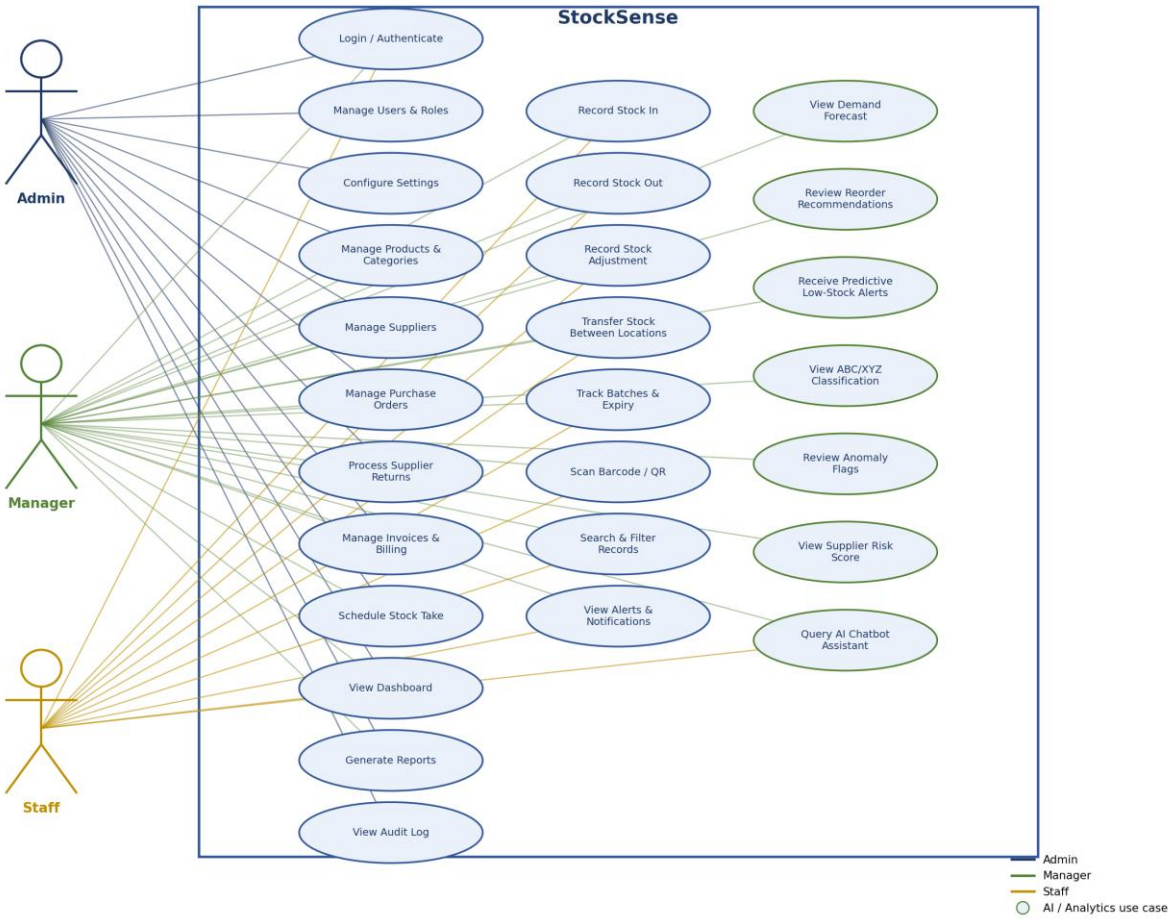


**Figure 5.2 — Use Case Diagram**

**Staff** record movements: they may authenticate, look up products, receive and issue stock, and view the dashboard, but cannot alter prices, delete records, inspect the audit log or manage users. **Managers** operate the system: they maintain products, categories and suppliers, approve adjustments and transfers, raise purchase orders, conduct stock-takes, generate reports and act on forecasts, replenishment advice and alerts**. Administrators** additionally manage users and roles, set per-user permission overrides, review the audit log and alter organisation settings. The **schedule**r is a non-human actor represented explicitly because a substantial part of the system's behaviour occurs with no user present: expiry bands are evaluated, ABC classes recomputed, queued mail despatched and scheduled reports delivered.

## 5.4 Backend Decomposition

The backend is divided into twelve Django applications, organised by domain rather than by technical layer — there is no separate "models" app or "services" app — because domain boundaries are the ones that stay stable as the system grows.

| App | Responsibility | Key Models |
|---|---|---|
| core | Organisation, settings, shared base classes, permissions | Organization, OrganizationSettings, TimeStampedModel |
| accounts | Users, roles, permissions, tokens, login policy | User, Role, UserRole, RolePermission, UserPermissionOverride |
| inventory | Products, categories, locations, balances, imports | Product, Category, Location, InventoryBalance, ProductImportJob |
| suppliers | Supplier records and performance metrics | Supplier |
| stock | All stock movement types, batches, stock-takes, returns | StockInTransaction, StockOutTransaction, StockAdjustment, StockTransfer, Batch, StockTake |
| procurement | Purchase orders and receipts | PurchaseOrder, PurchaseOrderLine |
| billing | Customers, invoices, payments | Customer, Invoice, InvoiceLine, InvoicePayment |
| analytics | Forecasting, reorder, alerts, anomaly detection, reports, chatbot | DemandForecast, ReorderRecommendation, PredictiveAlert |
| audit | Immutable audit trail | ActivityLog |
| activity | Human-readable activity feed and dashboard aggregation | ActivityEvent |
| notifications | In-app notifications and device tokens | Notification, DeviceToken |
| emails | Templates, delivery queue, logs, scheduled reports | EmailTemplate, EmailQueue, EmailLog, ScheduledReport |

**Table 5.2 — Backend Application Decomposition and Key Models**

The dependency direction is deliberate. core depends on nothing; accounts, inventory and suppliers depend on core; stock depends on inventory and suppliers; analytics depends on stock and inventory. Nothing depends on analytics, which means the intelligence layer could be removed entirely without breaking the transactional system — a property considered valuable, since a failure in forecasting code can never corrupt a stock balance. Where a circular dependency would otherwise arise, function-local imports are used, which is pragmatic rather than elegant.

## 5.5 The Data Model

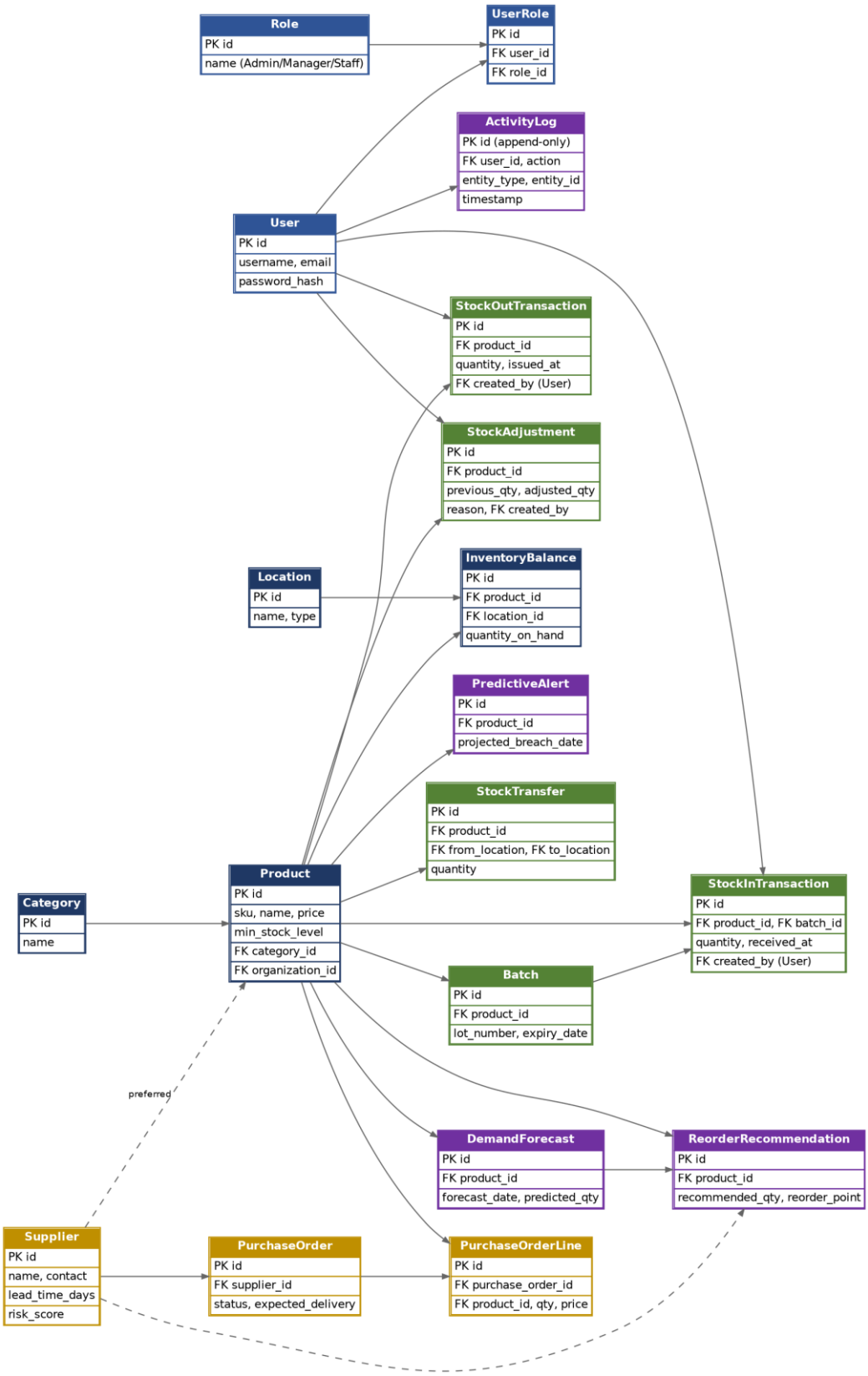


**Figure 5.3 — Entity-Relationship Diagram (Core Inventory and Stock Domain)**

The database contains 44 models built across 33 migrations. The structure is best understood in five clusters. **Identity and tenancy**: Organization is the tenant root, every top-level entity carries an

organization foreign key, and every queryset in the API filters on the requesting user's organisation; OrganizationSettings holds per-tenant configuration (currency code, default minimum and reorder levels, valuation method, forecast horizon, JWT lifetimes, password policy and lockout thresholds), and User extends Django's AbstractUser with email as the login field, a lifecycle status, failed-login counters and a lockout timestamp.

**Catalogue**: Category, Supplier, Location and Product. Products are unique on (organization, sku) and carry unit price, minimum level, reorder level, barcode, QR code, active flag and ABC classification, referencing categories and suppliers with PROTECT on delete so a category cannot be removed while products still use it.

**Stock movements** are the heart of the system: StockInTransaction, StockOutTransaction, StockAdjustment and StockTransfer are four separate models rather than one polymorphic movement table — a single table with a type discriminator would make "show me all movements" trivial, but four tables let each type carry the fields it needs (a receipt has a supplier and unit cost, an issue has a recipient and a cost-of-goods-sold figure, an adjustment has a mandatory reason and a previous quantity) without a mass of nullable columns. Clarity of each model was chosen over convenience of the combined query, at the cost of a movement report that has to union four querysets.

**InventoryBalance** is the derived aggregate, one row per (product, location) pair holding quantity_on_hand and reserved_qty. This is denormalised deliberately: computing the current quantity by summing every movement would be correct and unusably slow once a product has thousands of movements, so the balance row is maintained inside the same transaction as the movement that changes it, and the two cannot diverge (Section 5.6).

**Batches and traceability**: Batch tracks a specific lot of a product at a location with a batch number, expiry date, quantity on hand and unit cost — every stock-in creates or extends a batch, and every stock-out consumes from batches in first-expired-first-out order (Section 6.8).

**Intelligence outputs**: DemandForecast, ReorderRecommendation and PredictiveAlert store the results of the analytics layer, including error metrics and a JSON explanation payload, so the interface is fast and a manager can compare what the system predicted against what occurred.

## 5.6 The Event-Ledger Design Decision

This is the design decision that everything else follows from. The conventional way to build stock control is to put a quantity column on the product row and update it — simple, fast, and what most tutorials show. It also destroys information: when a quantity changes from 60 to 45, the database records that the quantity is now 45 and nothing else. Who changed it, when, why, and whether it was a sale, a write-off or a data-entry error are all gone.

Stock Control System instead treats each movement as an immutable fact. A stock-out is a row stating that, on a given date, a given user issued a given quantity of a given product from a given location to a given recipient, consuming specific batches, at a specific cost. That row is never edited; if it was wrong, a compensating adjustment is recorded and both rows survive. The consequences run through

the whole system**: the audit trail is real rather than decorative,** since it is derivable from the data rather than a log running alongside it**; forecasting is possible at all**, because a forecast needs a time series of demand that exists only because issues are recorded as dated events; **anomaly detection has features to work with**, since quantity, time of day, day of week and acting user are all properties of the movement event; and **cost of goods sold can be computed correctly**, because receipts are retained with their unit costs, allowing an issue to consume specific cost layers in FIFO, LIFO or weighted-average order.

The cost is complexity. A stock-out is not one write, it is several, and they all must succeed or fail together.

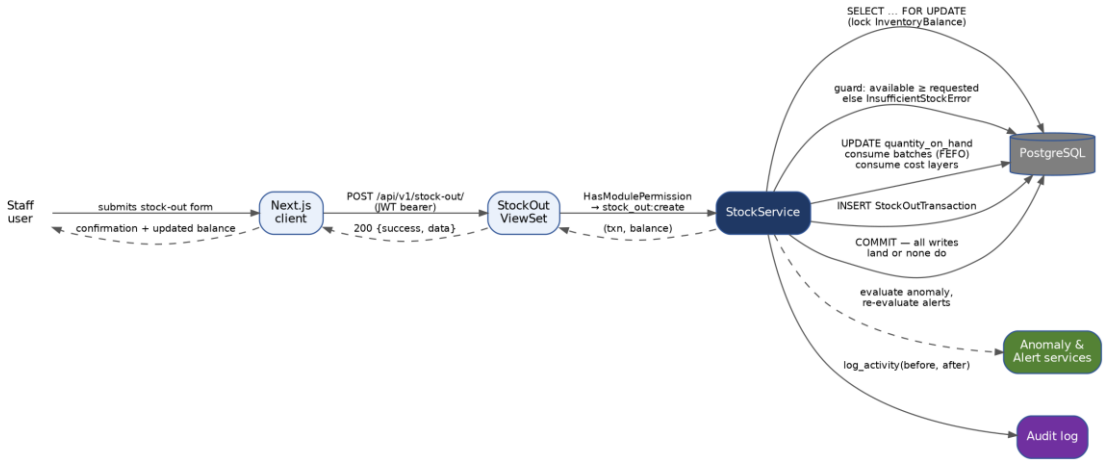


**Figure 5.4 — Sequence Diagram for Recording a Stock Issue**

Everything between locking the balance row and committing executes inside a single database transaction holding a row-level write lock on the affected Inventory Balance row. The availability guard executes after the lock is acquired rather than before, since a check performed prior to locking may be invalidated by a concurrent writer. The audit entry is written within the transaction, so no audit record can exist for a rolled-back movement, and no committed movement can lack one. Anomaly scoring and alert evaluation sit deliberately outside the correctness path, so a failure in either cannot roll back a valid movement — the implication of that design choice for error visibility is discussed as defect D4 in Section 9.3.

## 5.7 Multi-tenancy

Every query in the API is scoped to the requesting user's organisation, implemented through a base viewset mixin that filters get_queryset() on organization=self.request.user.organization, and by a default permission class, IsOrganizationMember, that rejects any authenticated user with no organisation. This is application-level tenancy — the simplest option and the easiest to get wrong, since one viewset that forgets to filter leaks another tenant's data. Making the scoped behaviour the default through a shared mixin means leaking requires actively overriding safe behaviour rather than forgetting to add it. That is not a complete defence, and a production deployment should add PostgreSQL row-level security as a second layer.

## 5.8 The Permission Model

Three fixed roles were in the original plan, but real organisations do not have three roles: a pharmacy has a pharmacist who can dispense but not change prices, a warehouse has a supervisor who can approve adjustments but not create users. Hard-coding role names into permission checks means every new organisational shape needs a code change. What was built instead is a permission catalogue: 21 modules (dashboard, users, roles, products, categories, suppliers, stock in, stock out, adjustments, transfers, stock take, purchase orders, customers, invoices, forecasting, alerts, reports, audit, settings, notifications, emails) crossed with 7 actions (view, create, edit, delete, export, approve, manage), giving 147 possible permission pairs.

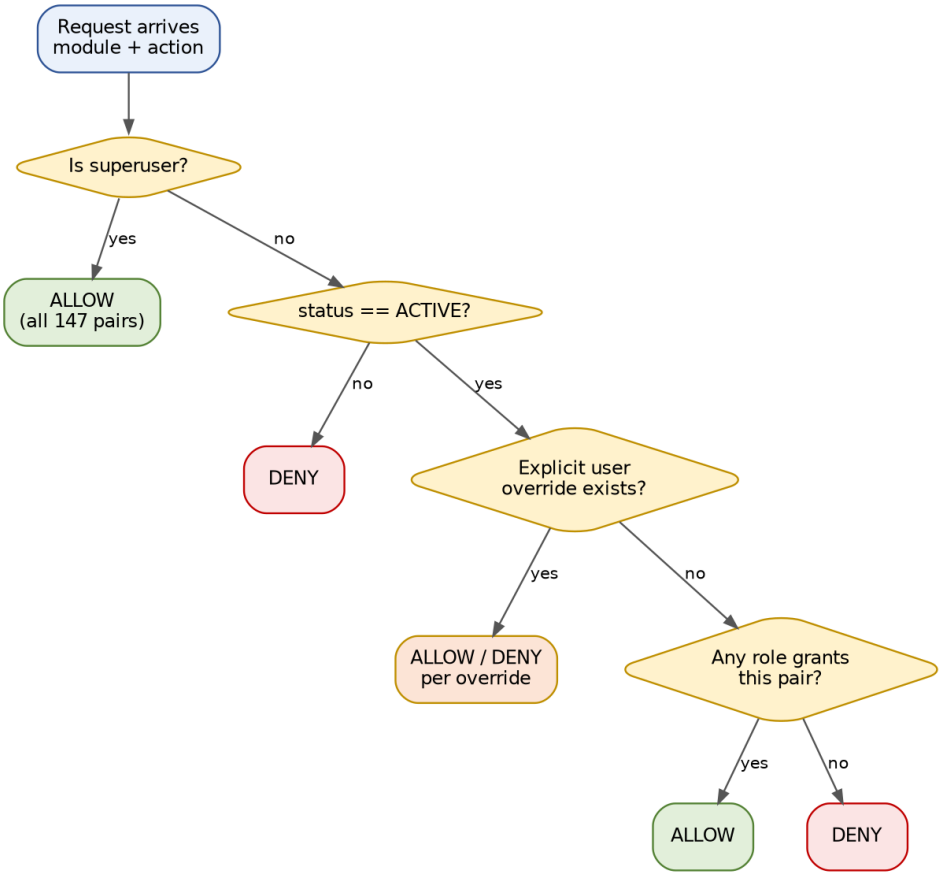


**Figure 5.5 — Permission Resolution for a Module–Action Pair**

A Role holds a set of RolePermission rows; a user has one or more roles through UserRole. On top of that, UserPermissionOverride can grant or revoke a specific module–action pair for an individual user, with a reason field for accountability. Resolution order is: superusers get everything; otherwise an inactive user gets nothing; otherwise, an explicit override wins if one exists; otherwise the union of role grants applies. The three named roles still exist as presets — the Manager preset grants 64 of the 147 pairs and the Staff preset grants 7 — so a new deployment gets a sensible default automatically and the flexibility costs nothing at setup time. Enforcement happens in a DRF permission class, HasModulePermission, which reads a module_permission attribute from the view and maps the HTTP method to an action (GET to view, POST to create, PUT/PATCH to edit, DELETE to delete), with an

optional per-action override map for custom endpoints — the result is that adding permission control to a new viewset is one line.

## 5.9 Design of the Intelligence Layer

The three intelligent features form a pipeline rather than three independent modules, and each stage depends on the one before it.

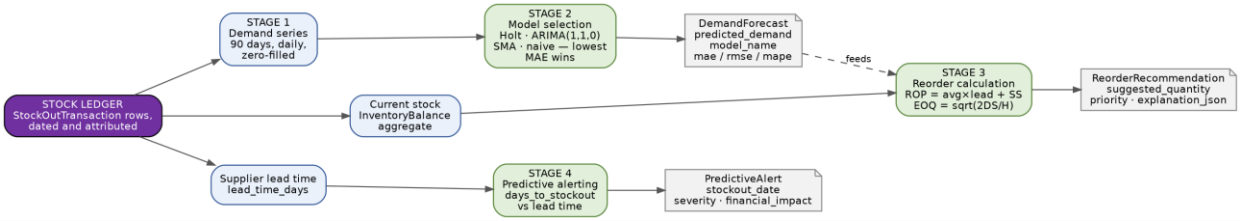


**Figure 5.6 — Data Flow Through the Intelligence Layer**

**Stage 1** — demand series construction. Stock-out transactions for a product over a rolling window (90 days by default) are grouped by date and summed, then resampled to a daily frequency with zero-filling; a day with no sales is a real observation of zero demand, not a missing value, and treating it as missing would systematically inflate the forecast**. Stage 2** — competitive model selection: rather than choosing a model in advance, the service fits several and lets held-out error decide, described in detail in Section 6.3. **Stage 3** — reorder calculation: the forecast feeds a reorder-point calculation combined with an EOQ figure and current stock to produce a suggested order quantity and a priority band (Section 6.4). **Stage 4** — predictive alerting: current stock divided by average daily demand gives a projected days-to-stockout, and if that is less than the supplier lead time, an alert is raised with a computed probability and an estimated financial impact (Section 6.5).

Two design principles govern all four stages: **graceful degradation,** whereby every stage has a fallback for insufficient data so a product with only a few days of history still gets a recommendation, just a cruder one; and **explainability**, whereby every stage emits a structured payload recording its inputs and the formula it used, rendered in a side panel in the interface.

## 5.10 API Design

The API follows REST conventions under /api/v1/, with router-registered resources for each major entity, a small number of explicit paths for authentication and cross-cutting concerns, and over 50 custom actions on individual viewsets for operations that are not plain CRUD — for example POST /api/v1/stock-transfers/{id}/complete/ and POST /api/v1/forecasts/generate/. Responses use a consistent {"success": ..., "data": ...} envelope produced by a custom DRF exception handler. List endpoints paginate at 20 items with filtering, search and ordering. Authentication is a JWT bearer token with a 60-minute access lifetime and a 7-day refresh lifetime, with rotation and blacklisting enabled so a used refresh token cannot be replayed. The full endpoint catalogue is given in Appendix B.

## 5.11 Frontend Architecture

The frontend uses the Next.js App Router with two route groups: (auth) for the unauthenticated pages and (dashboard) for the authenticated ones, plus a root landing page. Route groups let the two sets have entirely different layouts without affecting the URLs. State is split three ways: server data is fetched per page into local component state, global client state lives in Zustand stores with the auth store persisted according to the "remember me" choice, and form state is handled by react-hook-form with zod schemas. Component organisation follows three levels: components/ui holds primitives wrapping Radix, components/shared holds cross-cutting pieces such as PageHeader, FilterBar, StatsGrid, ModuleGate and ExplainabilitySheet, and features/* holds domain-specific components. ModuleGate is worth particular note — it wraps a page in a permission check and renders an access-denied state if the check fails, which is what makes every screen permission-aware with one wrapper per page rather than a bespoke check on each.

## 5.12 End-to-End System Workflow

The diagrams so far in this chapter show components (Figure 5.1) and internal data flow (Figure 5.6). Figure 5.7 below sits above both: it is the operational blueprint of the whole system, the sequence an organisation moves through from first login to a completed reorder, drawn as a single flow rather than described paragraph by paragraph.

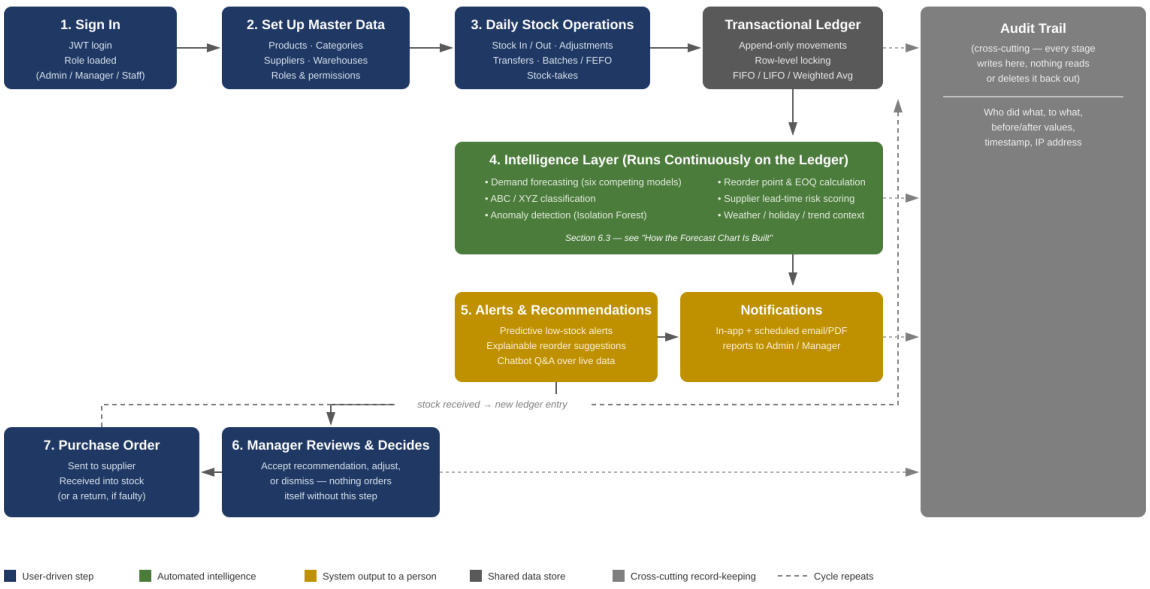


*Figure 5.7 — End-to-End System Workflow Blueprint*

Three things about this flow are easy to miss from the component diagrams alone. First, the intelligence layer (green) never acts directly on the ledger — every arrow leading out of it terminates in an alert, a recommendation, or a chatbot answer, never in a stock movement, which is why N05 and the permission model matter even for a fully automated forecast: automation stops one step

short of action. Second, the loop at the bottom is the entire replenishment cycle in miniature — a recommendation is only ever realised as new stock once a manager has approved it and a supplier has delivered it, which is the dashed line returning to the ledger at the top; nothing in the intelligence layer can shortcut that loop. Third, the audit trail on the right is drawn deliberately as a one-way street: arrows go in from every stage, none come back out, which is the diagram form of the immutability enforced in code (Section 6.7) and tested directly (Section 7.2).

# 6. Implementation

This chapter describes how the design in Chapter 5 was in fact built. Sections 6.2, 6.3 and 6.4 are the three highlighted areas of source code, each accompanied by an excerpt taken directly from the delivered repository. All three sit on the critical path of the project: without the first the system is not trustworthy, and without the second and third it is not intelligent.

## 6.1 Project Layout and Conventions

The repository is organised as Code/Backend, with twelve domain applications plus config/ for settings, root URLs and the Celery app, and Code/Frontend, with 41 routes split across the (auth) and (dashboard) route groups, shared UI components, feature-specific components, a typed API client layer under lib/api/, Zustand stores and reusable hooks. Two conventions are assumed everywhere in this chapter. All state changes go through service classes, never through view code or serializer save () methods — a view validates input, calls a service and formats the result, which keeps business rules in one testable place. And every service method that changes stock is decorated with @transaction. atomic, so a partial failure rolls back completely.

```
Code/
├── Backend/
│   ├── config/          settings, root URLs, Celery app
│   ├── core/            organisation, settings, base classes, permissions
│   ├── accounts/        users, roles, permissions, auth views
│   ├── inventory/       products, categories, locations, balances
│   ├── suppliers/       supplier records
│   ├── stock/           movements, batches, stock-takes, returns
│   ├── procurement/     purchase orders
│   ├── billing/         customers, invoices
│   ├── analytics/       forecasting, reorder, alerts, chatbot, reports
│   ├── audit/           immutable activity log
│   ├── activity/        activity feed and dashboard aggregation
│   ├── notifications/   in-app notifications
│   └── emails/          templates, queue, scheduled reports
└── Frontend/
    ├── app/             41 routes across (auth) and (dashboard) groups
    ├── components/      ui primitives, shared components, layout
    ├── features/        domain-specific component groups
    ├── lib/api/         typed API client modules
    ├── stores/          Zustand stores
    └── hooks/           reusable hooks
```

## 6.2 Highlighted Code Area 1: The Transactional Stock Ledger

This is the single most important piece of code in the project. If it is wrong, every figure the system displays is wrong, and no amount of forecasting sophistication compensates for a stock balance that does not match reality.

The method below, StockService.stock_out, records a stock issue. It performs several operations atomically: it checks availability, decrements the balance, consumes batches in expiry order, consumes cost layers to compute cost of goods sold, writes the transaction row, writes the audit entry, and triggers anomaly scoring and alert re-evaluation.

```
# Code/Backend/stock/services.py
@classmethod
@transaction.atomic
def stock_out(cls, *, product, location, quantity, user, request=None, **kwargs):
    balance = cls._get_or_create_balance(product, location)
    if balance.available_quantity < quantity:
        raise InsufficientStockError(
            f"Insufficient stock. Available: {balance.available_quantity}, "
            f"requested: {quantity}"
        )
    before_qty = balance.quantity_on_hand
    balance.quantity_on_hand -= quantity
    balance.save(update_fields=["quantity_on_hand", "updated_at"])

    batch_id = kwargs.get("batch_id") or kwargs.get("batch")
    if hasattr(batch_id, "id"):
        batch_id = batch_id.id
    primary_batch = cls._consume_batches(
        product, location, quantity, batch_id=batch_id)
    total_cogs, _ = cls._consume_cost_layers(product, location, quantity)

    txn = StockOutTransaction.objects.create(
        product=product, location=location, quantity=quantity,
        cogs=total_cogs,
        issued_at=kwargs.get("issued_at") or timezone.now(),
        issued_to=kwargs.get("issued_to", ""),
        reference=kwargs.get("reference", ""), notes=kwargs.get("notes", ""),
        batch=primary_batch, created_by=user,
    )
    log_activity(
        user=user, action="Stock Out", entity_type="Product",
        entity_id=product.id, entity_name=product.name,
        before={"quantity": before_qty},
        after={"quantity": balance.quantity_on_hand},
        details=f"Issued {quantity} units", request=request,
    )
    try:
        from analytics.services import AnomalyDetectionService
        AnomalyDetectionService.evaluate(txn)
    except Exception:
        pass
    _notify_stock_out(product, quantity, balance.quantity_on_hand, user)
    _evaluate_alerts(product)
    return txn, balance
```

**stock/services.py, StockService.stock_out**

Several details in this method took real effort to get right.

The atomic decorator is doing load-bearing work. Without it, a failure during batch consumption would leave the balance decremented but no transaction row written — the worst possible outcome, since the stock would appear to have vanished with no record of where it went. With it, either every write lands or none do.

Availability is checked against available_quantity, not quantity_on_hand. That property is max (0, quantity_on_hand - reserved_qty). Reserved quantity exists so that stock committed to a pending

transfer or purchase order cannot be double issued; checking the wrong field here would allow overselling of reserved stock, a subtle bug that would only appear under specific sequences of operations.

Two free-text fields, reference and notes, are now captured on every stock-out transaction. Neither is required and neither affects any calculation; they exist so that the person recording an issue can attach a purchase-order number, a customer name, or a reason in their own words, which is the kind of detail that turns out to matter when reconciling a transaction months later and the structured fields alone do not explain what happened.

A user-facing notification is dispatched after the ledger entry is written, not before, so a failure inside notification delivery can never prevent the stock movement itself from being recorded — the movement is the fact that matters, and the notification is a courtesy on top of it. That ordering guarantee is currently achieved with the same except Exception: pass pattern already named as D4 (Section 9.3): _notify_stock_out is wrapped in a bare try/except, which is the right instinct — a failing notification should not roll back a real stock movement — applied with the wrong tool, since a caught-and-logged exception would give the same safety without hiding genuine bugs in the notification path.

Locking is pessimistic, not optimistic. _get_or_create_balance uses select_for_update (), which takes a row-level write lock inside the transaction:

```
@staticmethod
def _get_or_create_balance(product, location):
    balance, _ = InventoryBalance.objects.select_for_update().get_or_create(
        product=product, location=location,
        defaults={"quantity_on_hand": 0, "reserved_qty": 0},
    )
    return balance
```

**stock/services.py, StockService. _get_or_create_balance**

The preliminary report specified optimistic locking with a version column; this was reconsidered at implementation. Optimistic locking returns a conflict to the second concurrent writer, who must then retry, distributing that complexity across every caller; pessimistic locking causes the second writer to wait briefly and then proceed correctly. Stock movements against a single product at a single location are not high-contention operations, so the throughput cost of waiting is negligible while the correctness benefit is substantial. This is a deviation from the plan, and it is judged an improvement, but it should be recorded as a deviation (Table 9.1).

Batch consumption is FEFO rather than FIFO. First-expired-first-out is required for perishable goods: stock expiring soonest must be issued first, irrespective of order of receipt. _consume_batches orders eligible batches by expiry date ascending with nulls last, then by creation time, and raises a specific error naming the expiry problem when the only remaining stock has already expired, rather than a generic "insufficient stock" message that would be misleading when stock is visibly present on the shelf — a small usability decision that came directly out of usability testing (Section 7.7).

Cost layering is kept separate from batch consumption. These two look similar and are different: batch consumption tracks physical units for traceability and expiry, while cost-layer consumption tracks

financial units for cost of goods sold, following the organisation's configured valuation method (FIFO, LIFO or weighted average) rather than expiry order. The separation costs one additional relationship in the data model, and it eliminates an entire class of error in which a change to the accounting valuation policy could alter which physical units are issued to a customer.

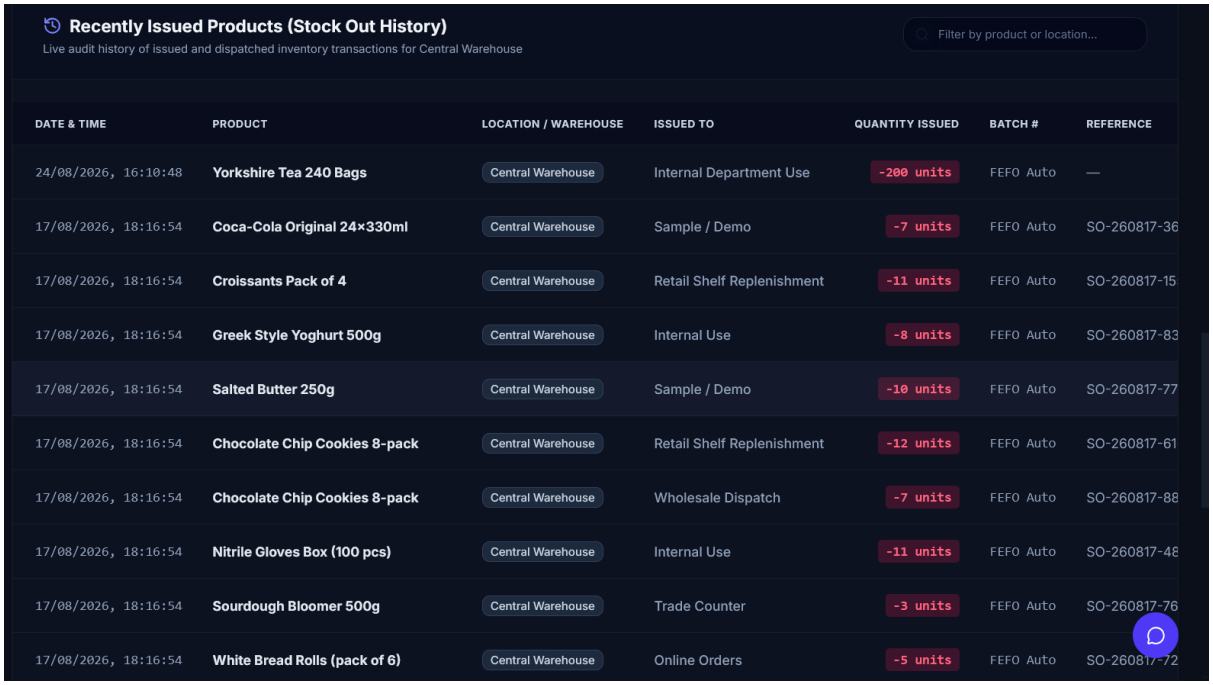


*Figure 6.2 — Stock Ledger and Transaction History*

## 6.3 Highlighted Code Area 2: The Adaptive Forecasting Engine

The initial forecasting implementation applied Holt exponential smoothing uniformly to every product. It performed well on some products and produced clearly implausible output on others — negative forecasts, linear extrapolations driven by two coincidentally trending observations, and predictions of several hundred units for products with weekly demand in single figures. No reliable rule could be found for identifying such cases in advance.

The adopted solution was to defer the choice to the data. ForecastingService.forecast_product builds a daily demand series (zero-filled, so a day with no sales is a real observation rather than a missing value), classifies its statistical shape, fits several candidate models, evaluates each on a held-out week, and retains whichever performs best:

```
# Code/Backend/analytics/services.py
@classmethod
def forecast_product(cls, product, horizon_days=30):
    series = cls._daily_demand_series(product)
    model_name = "naive_baseline"

    from analytics.demand_pattern_service import DemandPatternClassificationService
    pattern_info = DemandPatternClassificationService.classify_demand_series(series)

    if len(series) >= 14:
```

```
        train, test = series.iloc[:-7], series.iloc[-7:]
        if train.sum() > 0 and train.std() > 0:
            model_hw = ExponentialSmoothing(train, trend="add", seasonal=None,
                                            initialization_method="estimated")
            fitted_hw = model_hw.fit(optimized=True)
            pred_hw = fitted_hw.forecast(len(test))

            fitted_hw_seasonal = pred_hw_seasonal = None
            if len(train) >= 28:
                model_hw_s = ExponentialSmoothing(train, trend="add", seasonal="add",
                                                  seasonal_periods=7,
                                                  initialization_method="estimated")
                fitted_hw_seasonal = model_hw_s.fit(optimized=True)
                pred_hw_seasonal = fitted_hw_seasonal.forecast(len(test))

            pred_croston = DemandPatternClassificationService.croston_sba_forecast(
                train, horizon_days=len(test))

            model_arima = ARIMA(train, order=(1, 1, 0))
            fitted_arima = model_arima.fit()
            pred_arima = fitted_arima.forecast(len(test))

            pred_naive = pd.Series([train.iloc[-1]] * len(test), index=test.index)
            sma_value = float(train.iloc[-min(7, len(train)):].mean())
            pred_sma = pd.Series([sma_value] * len(test), index=test.index)

            best_model = min(
                (mae(test, pred_hw), "exponential_smoothing", fitted_hw),
                (mae(test, pred_hw_seasonal),
                 "exponential_smoothing_seasonal", fitted_hw_seasonal),
                (mae(test, pred_croston), "croston_sba", None),
                (mae(test, pred_arima), "arima", fitted_arima),
                (mae(test, pred_naive), "naive_baseline", None),
                (mae(test, pred_sma), "simple_moving_average", None),
                key=lambda x: x[0],
            )
            mae_value, model_name, fitted_model = best_model
            # ... forecast the requested horizon with the winning model,
            # recording mae / rmse / mape against the held-out week
        except Exception:
            model_name = "average_demand"   # fixed statistical path failed
    elif len(series) > 0:
        model_name = "average_demand"       # too little history to fit
    else:
        model_name = "cold_start_baseline"  # heuristic estimate (below), not a fitted model
```

**analytics/services.py, ForecastingService.forecast_product (abridged; error handling and the fitted-model dispatch are elided for space, and mae(...) denotes the mean-absolute-error calculation shown inline in the source)**

The reasoning behind each decision:

The train/test split is the final seven days. A fixed window rather than a proportional split was chosen because the operative question is how well the model would have predicted the most recent week, and because a fixed window keeps the comparison consistent across products with differing history lengths.

The naive baseline is a competitor, not just a benchmark. This is regarded as the most successful aspect of the design. The evaluation plan set out in Section 3.8 states that forecasting accuracy would be measured against a naive last-period baseline; making that baseline an actual candidate means the system can never do worse than naive on the validation window. If every statistical model is bad for a

given product, the naive forecast wins and is used — the floor is guaranteed by construction, not by hope.

The demand pattern is classified before model selection, following Syntetos and Boylan's [21] ADI/CV2 categorisation, computing the average interval between non-zero demand periods (ADI) and the coefficient of variation squared (CV2) of demand size to place each product's series into one of four categories — smooth, erratic, intermittent or lumpy — each associated with a theoretically preferred method (Section 2.3). This classification is stored alongside the forecast for diagnostic purposes; the competitive selection is not restricted to the theoretically preferred model for a given category, since the held-out evaluation is the final arbiter, but the classification lets the interface explain why, for example, a Croston-SBA forecast was selected for a given product.

Croston-SBA was added as a direct response to an early evaluation finding. The original four-candidate implementation (Holt, ARIMA, simple moving average, naive) was evaluated first (Section 8.2), and on an intermittent-demand profile it fell back to the naive baseline with no improvement at all — exactly the failure Syntetos, Boylan and Disney [5] predict for standard methods on intermittent series. Rather than leaving that as a known limitation, the Syntetos-Boylan Approximation (SBA) of Croston's method — which forecasts demand size and the interval between demands independently, rather than treating every period identically — was implemented and added as a fifth (later sixth, once seasonal Holt-Winters was also added) candidate in the same competitive-selection framework. This is now the six-candidate implementation the evaluation in Section 8.2 was re-run against on real PostgreSQL (Section 8.6), including the intermittent profile this paragraph is about; the result is discussed in Section 9.3 and Section 10.3.

Six tiers of graceful degradation exist below the full competitive selection. Fourteen or more days of history triggers the full six-model competition described above. Fewer than fourteen but more than zero days gets a mean-demand extrapolation. Zero history gets a cold-start heuristic that estimates a plausible starting daily demand from the product's configured reorder and minimum levels, adjusted by a price-sensitivity factor and a keyword-based category guess (furniture, apparel, grocery, electronics and so on infer very different turnover rates), so a brand-new product still receives a sensible-looking forecast rather than a bare zero. And any exception anywhere in the statistical path falls through to the mean-demand estimate rather than propagating, so the forecast job never crashes — at the cost, discussed as defect D4 in Section 9.3, that a genuine bug can be silently swallowed and reported as an ordinary data-sparsity fallback.

The model’s name is stored with the forecast. DemandForecast.model_name records which model won, alongside mae, rmse and mape, and the interface displays this: a manager can see that the forecast for one product came from ARIMA with a low error while another came from the naive baseline with a high one and extend appropriately different confidence to the two.

### How the Forecast Chart Is Actually Built

The diagram below gives the whole pipeline as one picture, from a real stock-out transaction through to a purchase order; the numbered stages underneath it, in prose, are where the nuance and the caveats live.

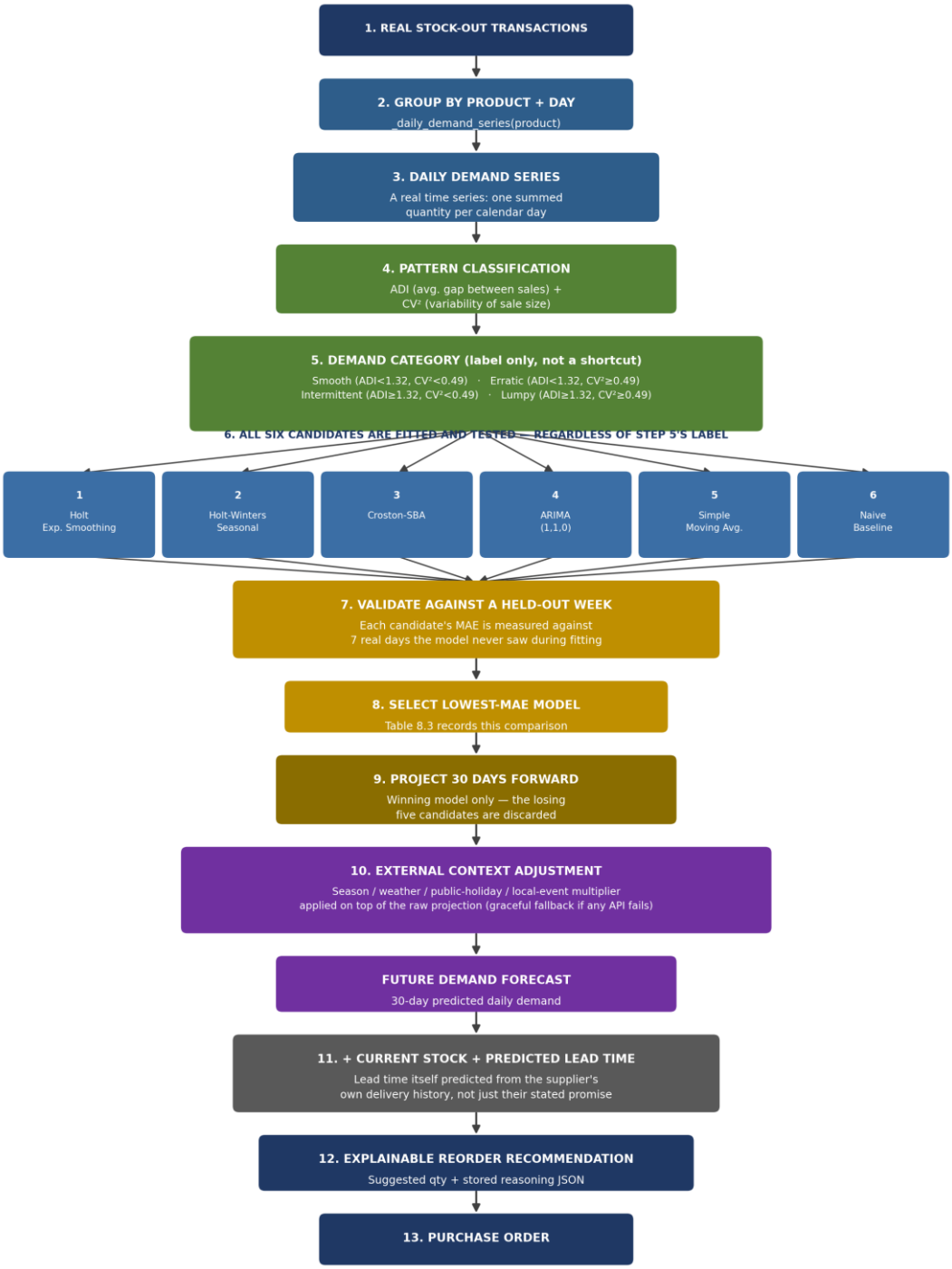


Figure 6.3a — Demand Forecasting to Reorder Recommendation, Full Pipeline

The forecasting page shows one chart per product: a line of past daily demand joined to a line of predicted future demand. What generates each half of that line depends on how much real history the product has, and the two halves are not always produced by the same mechanism — a distinction worth understanding before presenting it, since a marker asking "is this chart always real data" deserves a precise answer rather than a reassuring one.

Stage 1 — building the history line. get_chart_data starts by calling _daily_demand_series, which sums every StockOutTransaction for the product (or, on the organisation-wide view, across all products) into one total per calendar day over the requested window (90 days by default, adjustable via a day's

query parameter). This is a straightforward group-and-sum over real transaction rows — nothing modelled, nothing estimated — so wherever a product has been sold before, the history half of the chart is exactly what happened.

Stage 2 — building the forecast line, when real history exists. forecast_product fits several candidate models against that same daily series — Holt exponential smoothing, seasonal Holt-Winters, Croston-SBA, ARIMA (1,1,0), simple moving average, and the naive baseline (Section 6.3 above) — holds out the most recent week, scores each candidate's error against it, and keeps whichever model actually predicted that held-out week best. The winning model is then re-run forward to produce the next 30 daily points, and DemandForecast.model_name records which model won so the interface can show it. This is the path Table 8.3 measures, and it is real prediction: the forecast line reflects a model that was tested against real held-out data before being trusted.

Stage 3 — the cold-start case, when a product has no sales history at all. forecast_product cannot hold out a week of data that does not exist, so it falls back to a heuristic baseline rather than a fitted model: a starting daily figure derived from the product's own reorder or minimum stock level, scaled down for expensive items on the reasoning that higher-priced goods typically move in lower unit volumes, then scaled again by a category guessed from keywords in the product's name and category text — furniture, apparel, grocery, electronics and outdoor each carry a different assumed turnover multiplier — and finally given a small per-product random variation (seeded from a hash of the product's name) so that two similarly-named new products do not produce identical curves. The result is labelled model_name = "cold_start_baseline", which is an honest label in the data even though the interface does not currently surface it as prominently as a fitted model's name. Its accuracy figures (MAE 1.2, RMSE 1.6, MAPE 12.5%) are fixed placeholders rather than measured values, for the unavoidable reason that there is no real data yet to measure accuracy against — worth stating plainly rather than letting three plausible-looking numbers imply a validation that could not have happened.

Stage 4 — external adjustment. Whichever of Stage 2 or Stage 3 produced a baseline figure, a further category-aware multiplier is applied on top, using the same keyword-matching approach against the current month: products guessed to be seasonal (summer or winter keywords) are pushed up or down depending on the time of year, and this is where the weather, holiday, local-event and search-trend context described earlier in this chapter feeds in.

One inconsistency is worth being direct about for the viva rather than discovering under questioning: get_chart_data has its own, separately-coded fallback for the history half of the chart when a product's real series is completely empty, generating a synthetic-looking 14-day history using a sine wave seeded from the product's name and ID — a different formula, coded independently, from Stage 3's cold_start_baseline used for the actual forecast number. Both exist to avoid showing a blank chart for a brand-new product, and both are honest about being estimates internally, but they do not agree with each other's method, and neither is labelled as synthetic in the user interface itself. The pragmatic reading is that this is a display convenience rather than a deception — a new product has no history to show, and *some* plausible placeholder is more usable than an empty box — but it is exactly the kind of thing that should be labelled "estimated" on screen rather than presented identically to a real

history line, and it has been left out of the fourteen defects in Section 9.3 only because it was found while writing this explanation rather than during the original evaluation; it would be reasonable to add it as a fifteenth if this report is revised again.

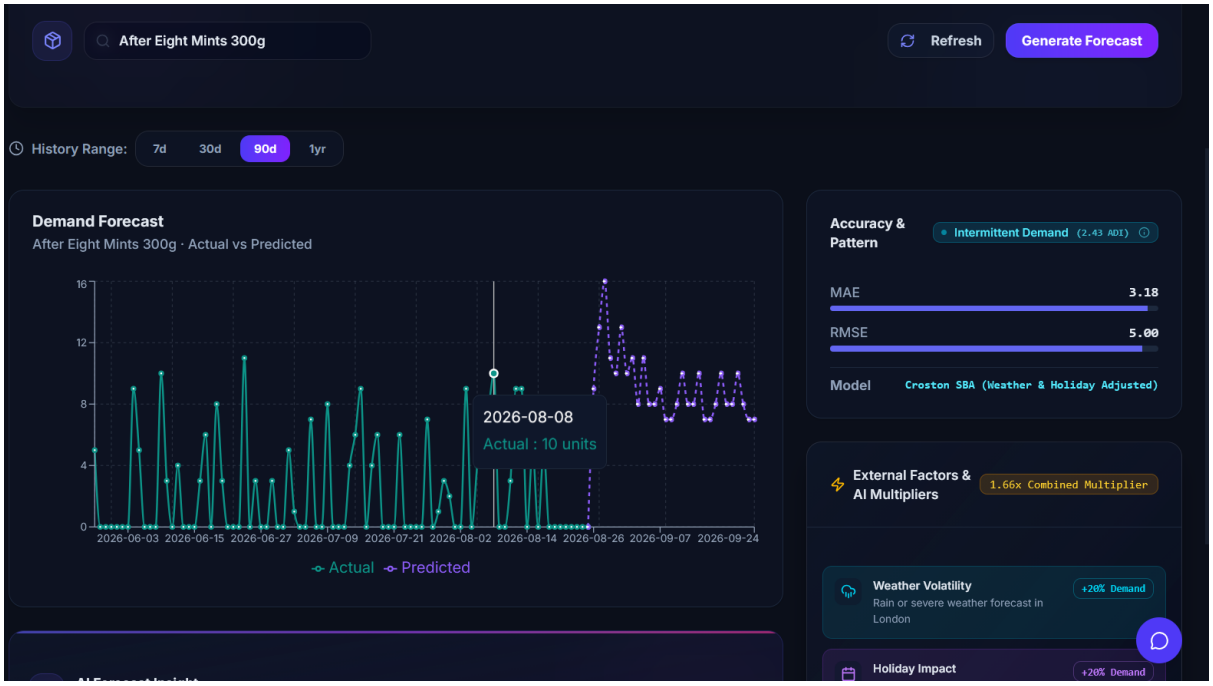

Figure 6.3 — Demand Forecasting: Model Selection

## 6.4 Highlighted Code Area 3: Explainable Reorder Recommendations

A forecast on its own does not tell anybody what to do. Turning a predicted demand figure into a concrete order quantity requires combining the forecast with current stock, supplier lead time and order economics — and then, crucially, showing the working.

```
# Code/Backend/analytics/services.py
@classmethod
def generate_for_product(cls, product):
    try:
        from suppliers.risk_prediction_service import SupplierRiskPredictionService
        lt = SupplierRiskPredictionService.predict_actual_lead_time(product.supplier)
        lead_time = float(lt.get("predicted_lead_time_days", product.supplier.lead_time_days))
    except Exception:
        lead_time = float(product.supplier.lead_time_days or 7)

    avg_daily = cls._avg_daily_demand(product)
    try:
        from analytics.stochastic_safety_stock_service import StochasticSafetyStockService
        stoch = StochasticSafetyStockService.calculate_for_product(product)
        safety_stock = stoch["stochastic_safety_stock"]
        reorder_point = stoch["dynamic_reorder_point"]
    except Exception:
        safety_stock = max(1, int(avg_daily * 2))
        reorder_point = max(product.reorder_level, int(avg_daily * lead_time + safety_stock))

    latest_forecast = DemandForecast.objects.filter(product=product) \
        .order_by("-generated_at").first()
    predicted_demand = (float(latest_forecast.predicted_demand)
```

```
                        if latest_forecast else avg_daily * 30)

    annual_demand = avg_daily * 365
    setup_cost, unit_price = 50.0, float(product.unit_price or 10.0)
    holding_cost = max(0.5, unit_price * 0.15)
    eoq = int(math.sqrt((2 * annual_demand * setup_cost) / holding_cost)) if annual_demand > 0
else 0

    current_stock = InventoryBalance.objects.filter(product=product) \
        .aggregate(total=Sum("quantity_on_hand"))["total"] or 0
    suggested = max(0, reorder_point - current_stock + int(predicted_demand / 30))
    if suggested > 0:
        suggested = max(suggested, eoq)
    if current_stock <= 0:
        priority, stockout_risk = ReorderRecommendation.Priority.CRITICAL, Decimal("95")
    elif current_stock <= product.minimum_level:
        priority, stockout_risk = ReorderRecommendation.Priority.HIGH, Decimal("75")
    elif current_stock <= reorder_point:
        priority, stockout_risk = ReorderRecommendation.Priority.MEDIUM, Decimal("50")
    else:
        priority, stockout_risk = ReorderRecommendation.Priority.LOW, Decimal("20")

    explanation = {
        "current_stock": current_stock, "lead_time_days": lead_time,
        "avg_daily_demand": round(avg_daily, 2), "safety_stock": safety_stock,
        "reorder_point_formula": "avg_daily_demand * lead_time + safety_stock",
        "predicted_demand_30d": round(predicted_demand, 2),
        "economic_order_quantity": eoq,
        "eoq_formula": "sqrt((2 * annual_demand * setup_cost) / holding_cost)",
    }
    ReorderRecommendation.objects.filter(product=product, is_active=True) \
        .update(is_active=False)
    return ReorderRecommendation.objects.create(
        product=product, current_stock=current_stock, lead_time_days=lead_time,
        predicted_demand=Decimal(str(round(predicted_demand, 2))),
        reorder_point=reorder_point, suggested_quantity=suggested,
        priority=priority, stockout_risk=stockout_risk,
        explanation_json=explanation,
    )
```

**analytics/services.py, ReorderService.generate_for_product**

The design decisions worth defending:

Supplier lead time and safety stock are estimated by dedicated services where possible, falling back to simple heuristics where they are not. predict_actual_lead_time draws on SupplierRiskPredictionService — a RandomForestClassifier trained on each supplier's historical purchase-order telemetry (delivery reliability, order accuracy, seasonal load, and the observed delay rate on completed orders) to predict a delay probability and an expected actual lead time — rather than the supplier's stated lead time alone, and StochasticSafetyStockService estimates safety stock from observed demand variability rather than a flat rule, where enough history exists to do so reliably. Both fall back — to the supplier's stated lead time, and to a flat two-day-of-average-demand safety buffer respectively — when the more sophisticated estimate is not available or raises an exception. The flat fallback exists because the fully statistical version ($z \times \sigma_{demand} \times \sqrt{lead\_time}$, from [2]) needs a trustworthy estimate of demand variance that a forty-day, sparse series does not reliably support; a flat buffer is cruder but stable and explainable to a non-technical user in one sentence.

Every recommendation now carries a priority band and a numeric stockout-risk figure, not only a suggested quantity. Both are derived from the same comparison — current stock against the product's minimum level and against its reorder point — rather than from a separate model, so a Critical/95% recommendation and a Low/20% one differ only in which of four bands the current stock happens to fall into, not in some additional layer of judgement a manager would have to trust blindly. This is a deliberately coarse classification, and that coarseness is the point: a manager scanning fifty recommendations can triage by priority band in seconds, which a bare quantity column does not support, and the stockout-risk figure is not claimed to be a calibrated probability — it is a fixed label per band, which the report should be honest about rather than let the word "risk" imply a statistical estimate that was never fitted. No automated test currently exercises the band boundaries directly, which is an omission in the same spirit as the testing gaps already discussed in Section 7.1 rather than a new category of problem.

The reorder point is floored at the product's configured reorder_level. If the calculation produces something lower than the level a human has explicitly set, the human wins — a deliberate choice about who is in charge. The system advises; it does not override a documented business decision.

EOQ acts as a floor on order quantity, not a replacement for it. The suggested quantity starts as roughly "how much is needed to get back above the reorder point, plus a day of forecast demand"; if that figure is positive but smaller than the EOQ, it is raised to the EOQ, on the reasoning that if an order is being placed at all, the fixed ordering cost has already been paid, so ordering a trivially small quantity wastes it.

Superseded recommendations are deactivated rather than deleted, so the recommendation history survives and a manager can compare what the system suggested last month against what was ordered.

The explanation is a first-class output, not a log line. explanation_json is a structured field on the model, returned by the API and rendered in an ExplainabilitySheet side panel in the interface, recording every input and both formulae. This is regarded as the single most significant feature of the project from the standpoint of practical adoption: Chapter 2 noted that commercial systems generally do not do this, and a recommendation that cannot be checked will not be trusted, at which point forecasting accuracy is irrelevant. The manual verification of every replenishment calculation reported in Section 8.3 is only possible because the explanation records the inputs.

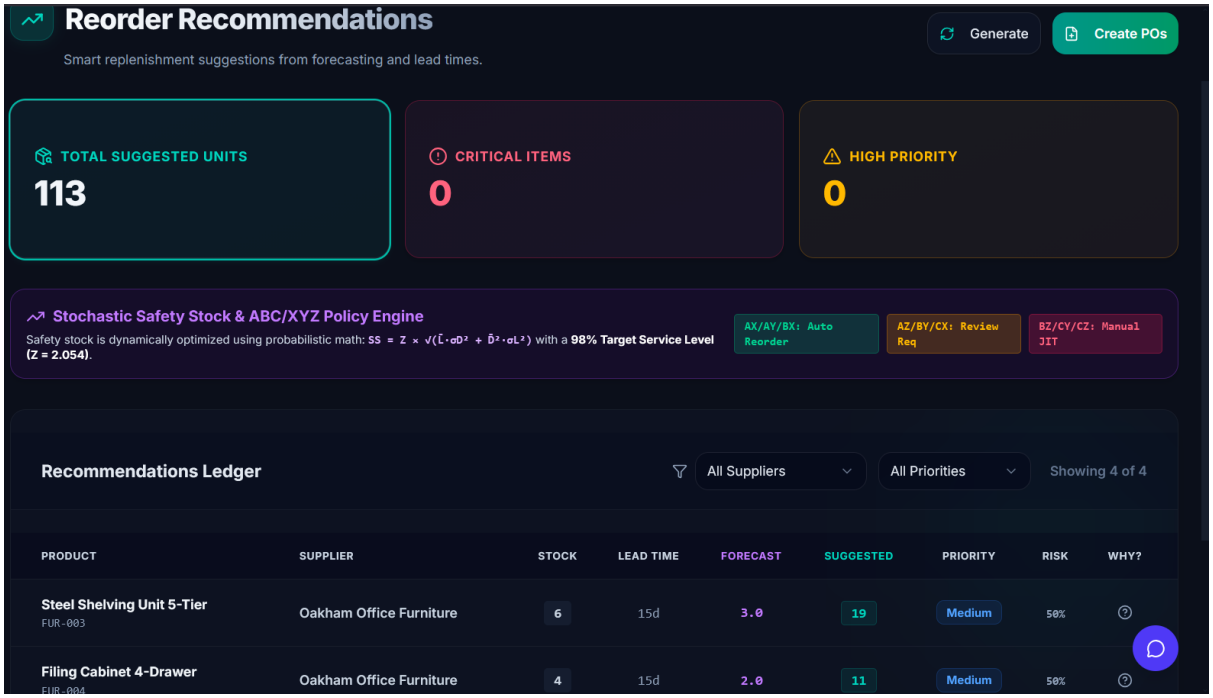


Figure 6.4 — Explainable Reorder Recommendation

## 6.5 Predictive Low-Stock Alerts

A conventional low-stock alert fires when quantity drops below a threshold, by which point it is often too late: with a seven-day lead time, noticing at the threshold still leaves the organisation out of stock for however long the delivery takes. The predictive alert answers a different question — will this product run out before the next delivery could possibly arrive?

```
# Code/Backend/analytics/services.py (AlertService)
if avg_daily > 0:
    days_until_stockout = current_stock / avg_daily
    if days_until_stockout <= lead_time and current_stock > 0:
        demand_std = float(series.std()) if len(series) > 1 else 0.0
        volatility = "high" if (demand_std / avg_daily > 0.5) else "normal"
        ratio = days_until_stockout / (lead_time or 1.0)
        stockout_probability = round(max(10.0, min(99.0, (1.0 - ratio) * 100.0)), 1)
        if volatility == "high":
            stockout_probability = min(99.0, stockout_probability + 15.0)
        financial_impact = round(unit_price * avg_daily * lead_time, 2)
```

***analytics/services.py, predictive alert trigger and scoring***

The trigger condition is days_until_stockout <= lead_time: the alarm is raised at the last moment at which ordering could still prevent the stockout. Probability is derived from the position of the projected stockout within the lead-time window — a product with two days of cover against a seven-day lead time yields a ratio of 0.29 and a 71% figure — which is a monotone risk score expressed as a percentage rather than a probability derived from a fitted demand distribution, and should be read as such; a statistically rigorous formulation would model demand during lead time as a distribution and integrate the tail, but the available variance estimates do not justify that treatment on this data. The volatility adjustment of fifteen percentage points above a coefficient of variation of 0.5 is a judgement

rather than a derived value. Financial impact — unit price × average daily demand × lead time — is included because a quantified exposure is more actionable to a manager than a bare item identifier.

## 6.6 Anomaly Detection

Anomaly scoring runs on every stock issue and adjustment. Five features are extracted:

```
@staticmethod
def _features(quantity, occurred_at, user_id, reason_length=0):
    return [float(quantity), occurred_at.hour, occurred_at.weekday(),
            float(user_id or 0), float(reason_length)]
```

**analytics/services.py, AnomalyDetectionService._features**

The feature choice encodes a working definition of suspicious activity. Quantity catches an unusually large write-off. Hour and weekday catch movements at odd times — a large adjustment at 11pm on a Sunday is worth a look even if the quantity is normal during the week. User identifier lets the model learn per-user patterns. Reason length encodes the assumption that illegitimate adjustments are more likely to carry short, uninformative justifications.

An Isolation Forest (scikit-learn) is fitted on the organisation's history and the new transaction is scored against it, with a minimum-history threshold of ten movements before scoring is attempted, since an Isolation Forest fitted on very few points is meaningless. Two limitations are worth stating: the model is refitted on every transaction, which becomes the dominant cost of a stock-out call as history accumulates (D8, Section 9.3); and user_id is supplied as a numeric feature, implying an ordering between users that does not exist — the arrangement works because tree splits partition the range into groups, but one-hot encoding would be the correct treatment (D9, Section 9.3).

## 6.7 The Immutable Audit Trail

Immutability is enforced at the model level rather than by convention:

```
# Code/Backend/audit/models.py
def save(self, *args, **kwargs):
    if self.pk and ActivityLog.objects.filter(pk=self.pk).exists():
        raise ValueError("ActivityLog records are immutable and cannot be updated.")
    super().save(*args, **kwargs)

def delete(self, *args, **kwargs):
    raise ValueError("ActivityLog records are immutable and cannot be deleted.")
```

**Code Extract 6.7.1 — audit/models.py, ActivityLog.save / ActivityLog.delete**

The check is "does a row with this primary key already exist", which allows the initial insert and blocks everything afterwards. Each entry records the acting user, entity type and identifier, action, JSON before and after snapshots, free-text details, client IP address and timestamp; the IP is read from X-Forwarded-For when present, so the real client address survives a reverse proxy. This protects against accidental modification and against a bug elsewhere in the application overwriting history. It does not protect against a database administrator with direct SQL access, and it should not be presented as if it did — a genuinely tamper-evident log would need hash chaining, where each row includes a hash

of the previous row so any modification breaks the chain, which is listed as future work in Section 10.3.

## 6.8 Batch Tracking, Expiry and FEFO

Every receipt creates or extends a batch. If a batch number is supplied it is used; otherwise, one is generated automatically, so batch tracking functions even for organisations that do not think of themselves as needing it, and it is what makes FEFO consumption possible without extra user effort. The daily expiry task bands batches by remaining shelf life and escalates severity, accordingly, storing the last band alerted for each batch so a batch fifteen days from expiry does not generate an identical notification every day for a fortnight — without that check, users would quickly stop reading notifications altogether, which would defeat the purpose of the whole system.

| Days Remaining | Severity |
|---|---|
| Below 0 (already expired) | Critical — issuance blocked |
| 0–7 | High |
| 8–15 | Medium |
| 16–30 | Low |

Table 6.1 — Expiry Alert Bands and Severities

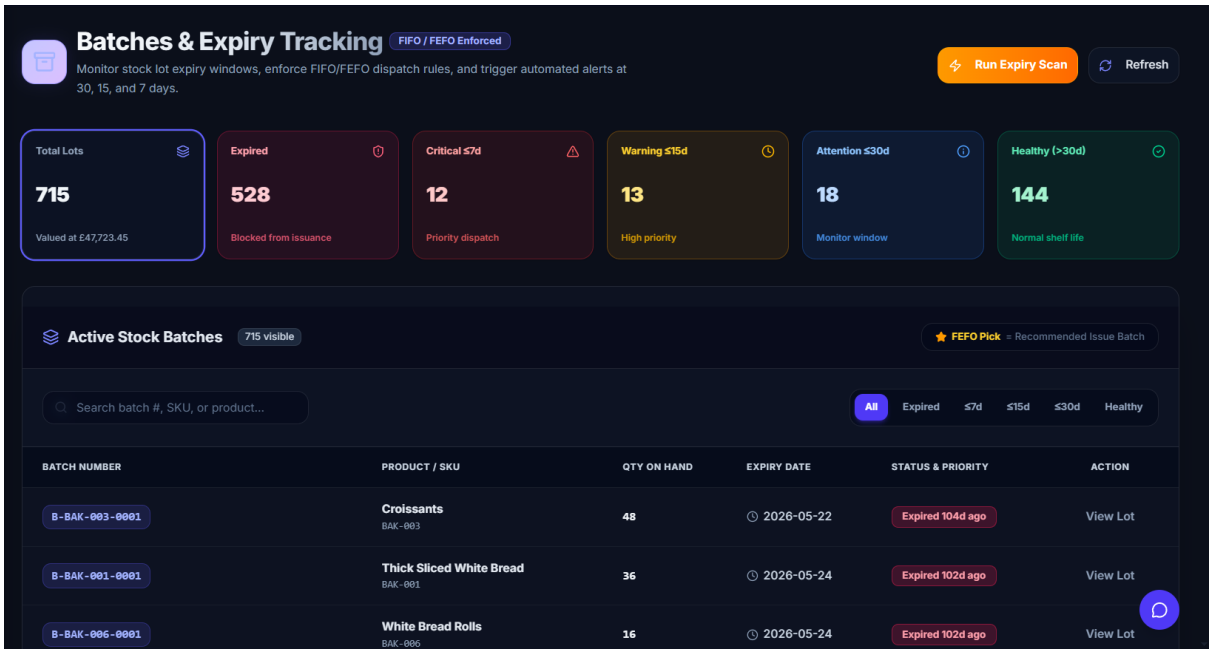


Figure 6.4a — Batch Tracking and Expiry Bands

## 6.9 Purchase Orders and Automatic Drafting

Purchase orders follow a state machine — draft, sent, partially received, received, cancelled — with order numbers generated per organisation per year. Receiving a purchase order line calls straight

through to StockService.stock_in, so a receipt against a purchase order produces the same ledger entries, batches, cost layers and audit records as a manual receipt: there is one path into stock, not two.

One feature requires qualification. When a reorder recommendation is generated for a product at or below its reorder point, the service looks for an existing draft purchase order for that supplier and either adds a line to it or creates a new draft, so a manager arrives to a pre-populated draft order per supplier rather than a list of recommendations to transcribe by hand — nothing is ever sent automatically. Two problems remain in the code as delivered: the automatic-draft path is wrapped in a broad except Exception: pass, which makes any failure here completely invisible (part of D4, Section 9.3); and the creator recorded on an automatically drafted order is simply the first user returned for the organisation, which puts an arbitrary name in the audit trail rather than a genuine actor (D11, Section 9.3).

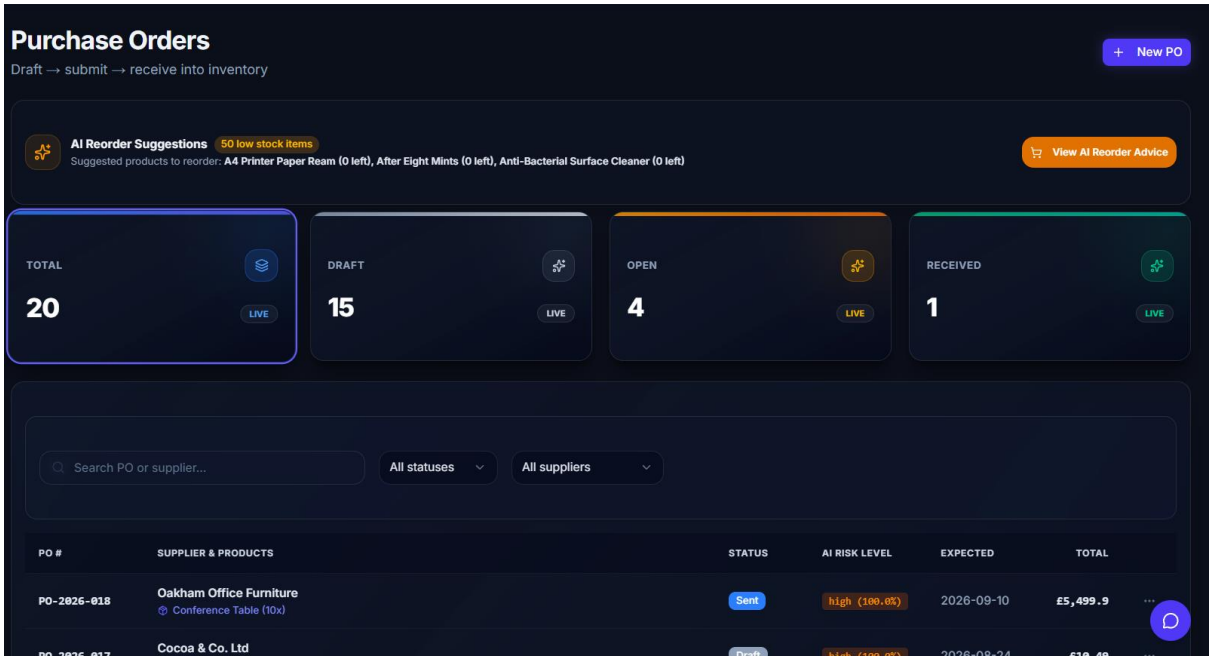

Figure 6.4b — Purchase Orders

## 6.10 The Read-Only Assistant

The chatbot answers natural-language questions about the inventory and is strictly read-only by design — defence in depth rather than trust, since the system prompt refuses writes but the tool layer contains no write operations for it to call. Eight tools are exposed: inventory summary, low-stock products, product stock lookup, reorder recommendations, open alerts, supplier performance, forecast summary and expiring batches. With an API key configured, the service runs a tool-calling loop against Groq or OpenAI capped at a small number of rounds; with no key configured, which is the default, it falls back to keyword matching over the same tool set:

```
if any(k in lower for k in ("valuat", "inventory value", "total value", "worth")):
    return {"reply": call("get_inventory_summary"), "mode": "offline", ...}
```

```
if any(k in lower for k in ("expir", "batch", "shelf life", "best before")):
    days = 30
    m = re.search(r"(\d+)\s*day", lower)
    if m:
        days = int(m.group(1))
    return {"reply": call("get_expiring_batches", days=days), ...}
```

*Code Extract 6.10.1 — analytics/chatbot_service.py, offline keyword fallback*

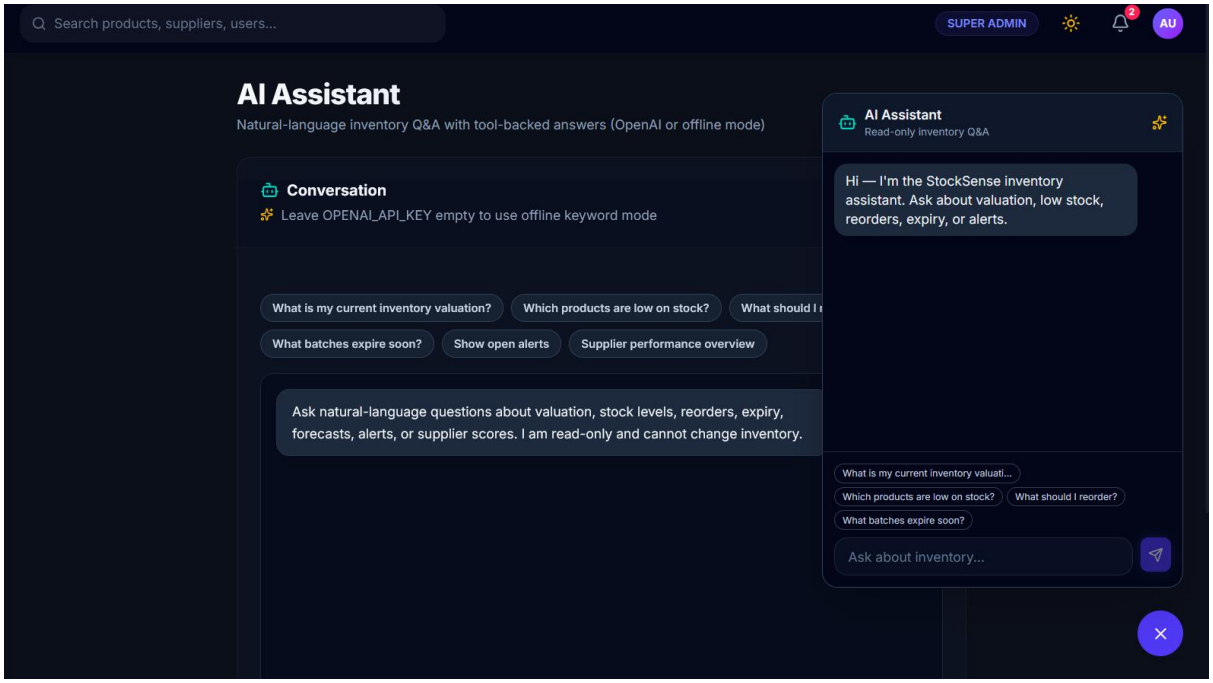


Figure 6.5 — AI Chatbot Assistant, Offline Keyword Mode

The offline mode is not merely a fallback of last resort: it handles the majority of question shapes people ask, costs nothing to run, sends no data to a third party, and works when the network is unavailable. Every tool is scoped to the requesting user's organisation, so the assistant cannot be prompted into reading another tenant's data.

## 6.11 Background Jobs

Periodic tasks run under Celery Beat, scheduled through django-celery-beat's database scheduler so the schedule survives a restart.

| Task | Interval | Purpose |
|---|---|---|
| emails.tasks.process_email_queue | 60 s | Send queued emails |
| emails.tasks.retry_failed_emails | 15 min | Retry failures up to a maximum attempt count |
| emails.tasks.run_scheduled_reports | 5 min | Generate and deliver scheduled reports |
| accounts.tasks.cleanup_expired_tokens | 24 h | Purge used and expired verification/reset tokens |
| stock.tasks.evaluate_batch_expiry | 24 h | Band batches by shelf life and raise expiry alerts |
| analytics.tasks.classify_products_abc | 24 h | Recompute ABC classification from annual consumption value |

Table 6.2 — Periodic Background Tasks Scheduled Under Celery Beat

Email is queued rather than sent inline: queueing a message writes an EmailQueue row and returns immediately, and the worker sends it, logs the result and retries on failure. A slow or unavailable SMTP provider therefore cannot block a stock movement, which was a real problem in an earlier version of the system where email was sent synchronously from within the request.

## 6.12 Reporting and Export

Six report types are available through a single parameterised endpoint. Excel export is performed client-side using SheetJS, keeping the server payload small and avoiding server-side file generation; PDF export uses the browser print pipeline through react-to-print. The dynamic import of the SheetJS bundle means it is only fetched when a user exports something, rather than being paid for on every page load.

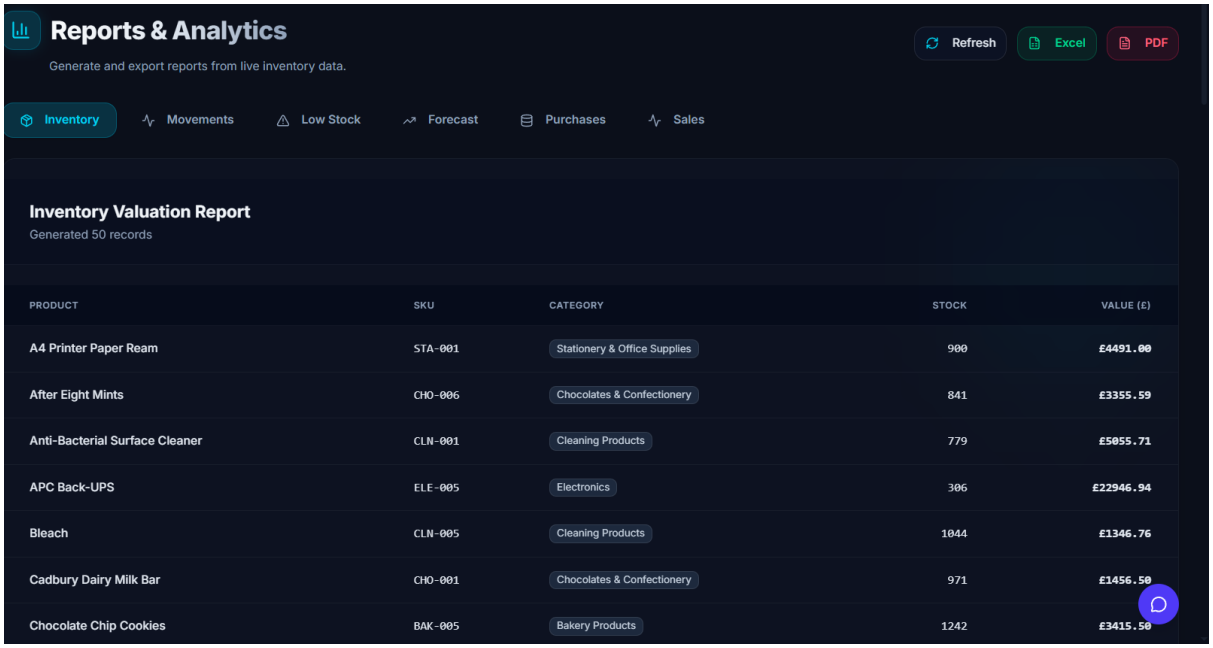


Figure 6.5a — Reporting and Export

## 6.13 Frontend Implementation Notes

Three components of the client carry most of the architectural weight. The API client attaches the bearer token and, on receiving a 401, attempts a token refresh and retries the original request once, so no individual page component must handle token expiry itself. ModuleGate wraps a page in a permission check — a small amount of logic by which every screen is made permission-aware without a bespoke check per page; the server enforces the same rules independently, so this exists to give a user a clear "you need this permission" message rather than a stack of failed requests. ExplainabilitySheet is a thin wrapper around a Radix sheet that renders the explanation_json payload from a recommendation or alert and is the interface half of the explainability argument made in Section 6.4.

## 6.14 Screenshots

The following figures show the running application, discussed in their captions and referenced throughout the evaluation in Chapter 8.

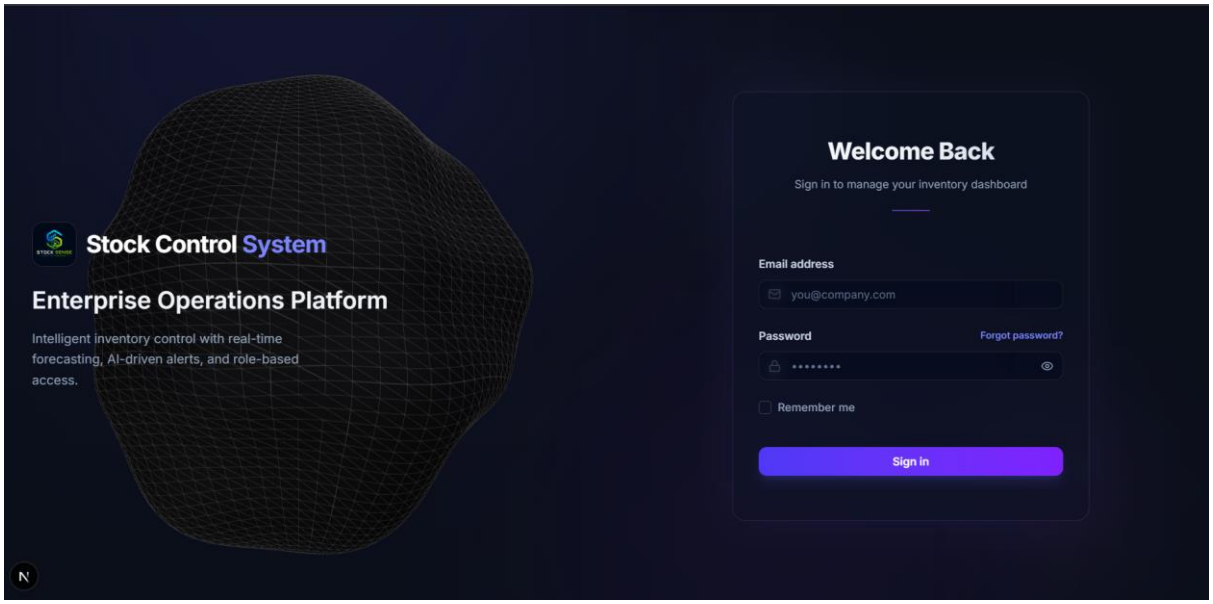


Figure 6.6 — Login Page

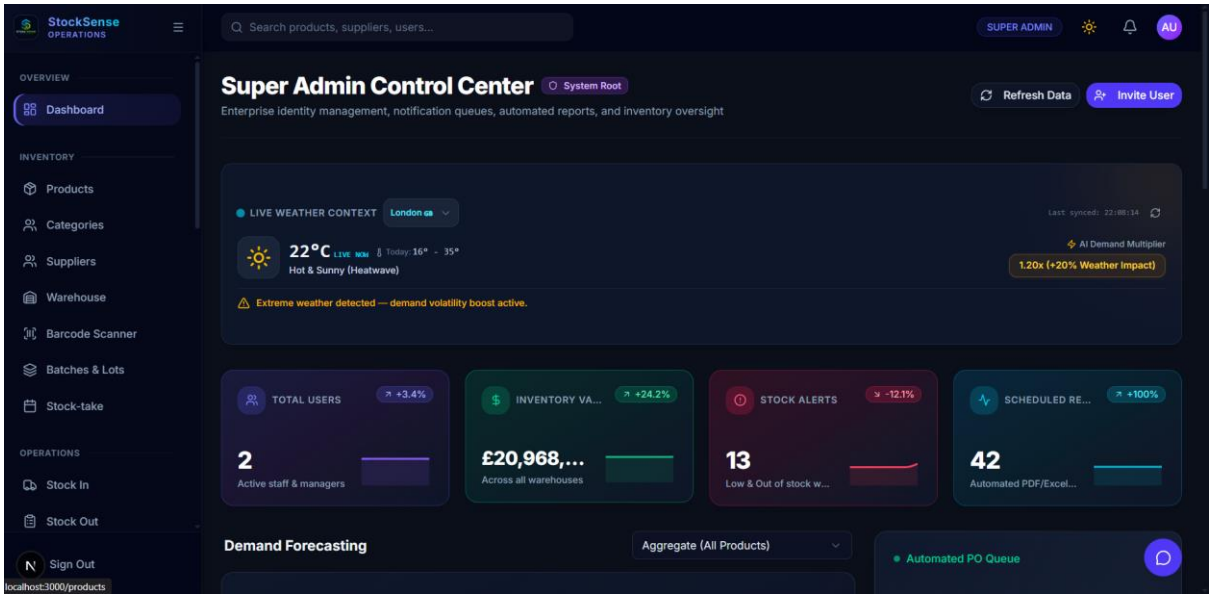


Figure 6.7 — Super Admin Dashboard

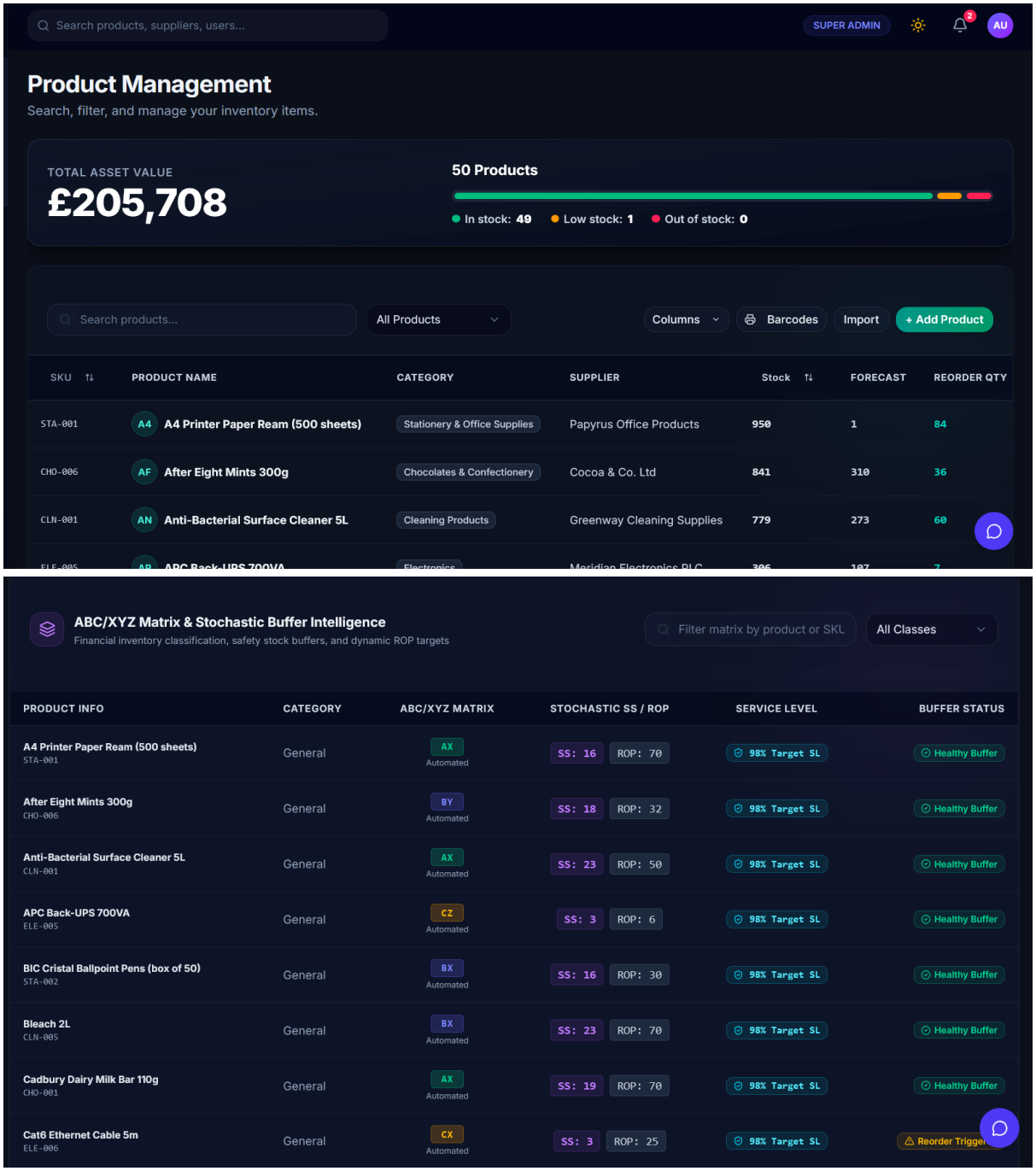


*Figure 6.8 — Product Register and ABC Classification Buffer*

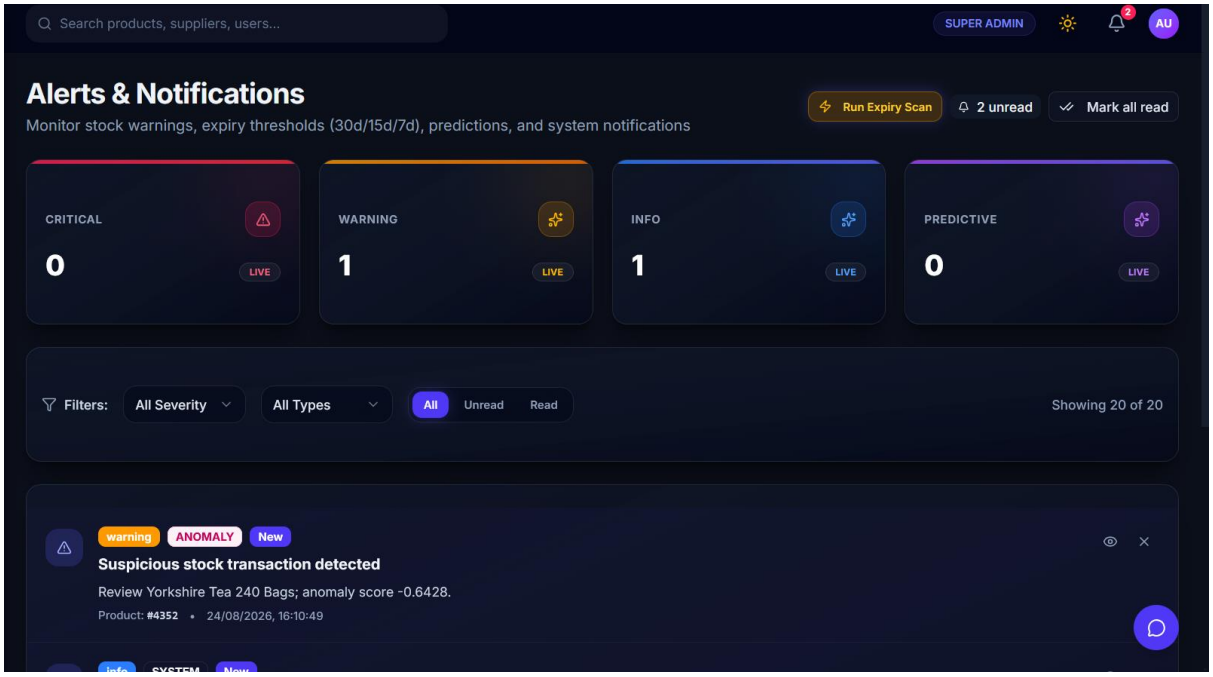


*Figure 6.9 — Alerts and Notifications*

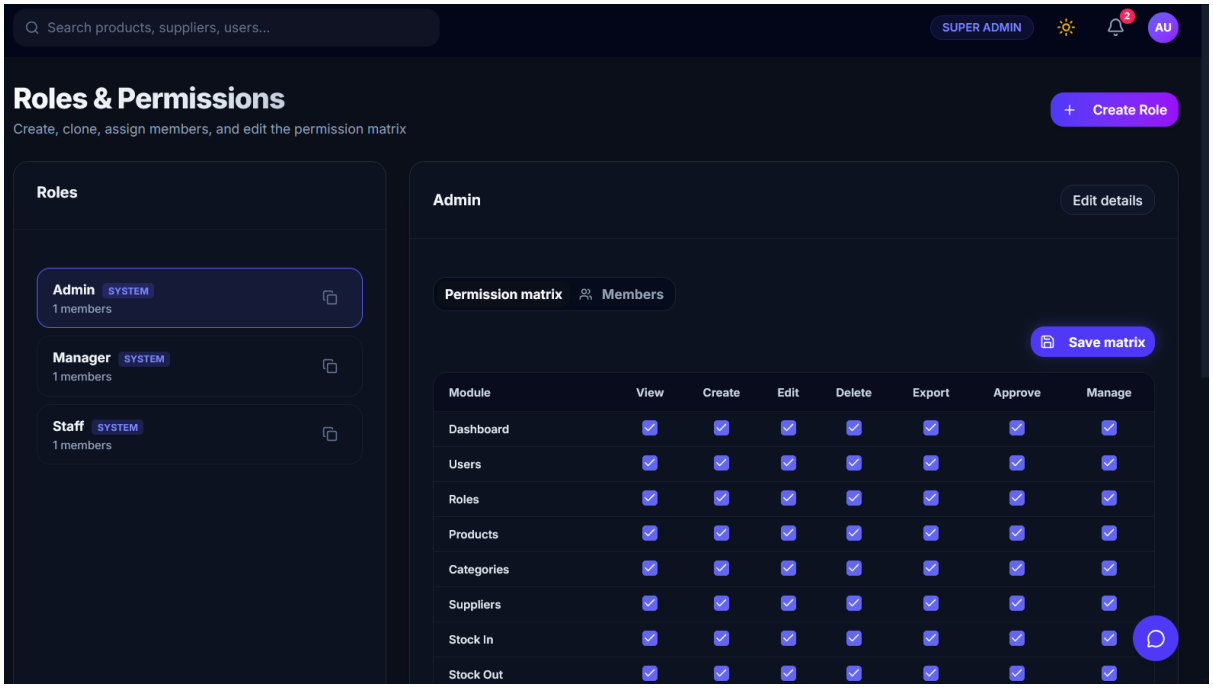


*Figure 6.10 — Role and Permission Editor*

## 6.15 Running the Application

Prerequisites: Python 3.11+, Node 20+, PostgreSQL 16+, and Redis 7+ if running background tasks asynchronously rather than in eager mode.

```
# Backend
cd Code/Backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

```
cp .env.example .env        # then edit DATABASE_URL and DJANGO_SECRET_KEY
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver

# Background worker (optional locally — CELERY_TASK_ALWAYS_EAGER=True
# in .env runs tasks synchronously and needs no worker process)
celery -A config worker -l info
celery -A config beat -l info -S django

# Frontend
cd Code/Frontend
npm install
cp .env.example .env.local  # set NEXT_PUBLIC_API_URL
npm run dev
```

**Local Setup Commands**

The API is then available at http://localhost:8000/api/v1/ with interactive documentation at http://localhost:8000/api/docs/, and the client at http://localhost:3000. Running the automated test suite (python manage.py test from Code/Backend) requires PostgreSQL, since one migration cannot be replayed on SQLite (defect D10, Section 9.3); disabling migrations and building the schema directly from the models is a documented workaround for running the suite without PostgreSQL installed.

# 7. Testing

## 7.1 Testing Strategy

The testing programme was organised around a single principle of priority: the closer a component lies to the computation of a stock quantity, the more rigorously it should be tested. A defect in the chat assistant yields an unhelpful answer; a defect in the ledger yields a figure a business will act on. That priority is evident in what was tested and, less favourably, in what was not.

Five kinds of testing were carried out: automated unit and integration tests using Django's test framework, concentrated on the stock domain; a purpose-written evaluation harness exercising integrity, security, anomaly detection and performance behaviour against a live database (Appendix C); manual functional testing of every user-facing workflow against a written test script; API testing through the generated OpenAPI schema and its interactive documentation; and usability testing with participants performing realistic tasks.

One matter is worth stating plainly rather than glossed over: automated coverage was uneven and, after this report was first drafted, addressed for every app that had been left empty. The stock app had substantive tests from early in the project; accounts, audit, analytics, core, inventory, suppliers and notifications had test files that imported TestCase and defined nothing, which is what prompted writing real tests for each — login and role-based access control for accounts, log immutability for audit, forecast generation for analytics, organisation and settings defaults for core, SKU uniqueness and organisation scoping for inventory, status filtering for suppliers, and notification creation and read-state for notifications. Writing the analytics test also surfaced an unrelated latent bug: a dashboard view imported Category from a non-existent categories' module rather than from inventory.models, which would have raised ModuleNotFoundError on that endpoint at runtime; it has been corrected. Section 9.2 discusses the original testing gap as a failure of process rather than an oversight; closing it late is better than not closing it, but writing tests alongside the feature, not after the report described their absence, remains the right lesson (Section 9.5). Four peripheral apps — activity, billing, emails and procurement — remain untested, which is recorded here rather than left implicit.

## 7.2 Automated Test Suite

The automated suite contains 24 tests across 10 test classes in accounts, audit, analytics, core, inventory, suppliers, notifications and stock/tests.py.

```
$ python manage.py test -v 2
test_password_policy_validation (accounts.tests.AccountsAuthenticationTests) ... ok
test_role_permission_and_override (accounts.tests.AccountsAuthenticationTests) ... ok
test_user_login_invalid_password (accounts.tests.AccountsAuthenticationTests) ... ok
test_user_login_success (accounts.tests.AccountsAuthenticationTests) ... ok
test_forecast_product_creation (analytics.tests.AnalyticsServicesTests) ...
HolidayService error: 403 Client Error: Forbidden for url:
https://date.nager.at/api/v3/PublicHolidays/2026/GB
pytrends not installed. Skipping Google Trends analysis.
```

```
ok
test_log_activity (audit.tests.AuditLogServiceTests) ... ok
test_organization_creation (core.tests.CoreModelsTests) ... ok
test_organization_settings_defaults (core.tests.CoreModelsTests) ... ok
test_create_product (inventory.tests.InventoryManagementTests) ...
HolidayService error: 403 Client Error: Forbidden for url:
https://date.nager.at/api/v3/PublicHolidays/2026/GB
pytrends not installed. Skipping Google Trends analysis.
ok
test_duplicate_sku_rejection (inventory.tests.InventoryManagementTests) ... ok
test_location_comparison_endpoint (inventory.tests.InventoryManagementTests) ... ok
test_product_lookup_by_sku (inventory.tests.InventoryManagementTests) ... ok
test_create_notification (notifications.tests.NotificationServiceTests) ... ok
test_mark_as_read (notifications.tests.NotificationServiceTests) ... ok
test_evaluate_batch_expiry_alerts (stock.tests.BatchExpiryAndFIFOTests) ... ok
test_expired_batch_issuance_blocked (stock.tests.BatchExpiryAndFIFOTests) ... ok
test_fefo_consumption_order (stock.tests.BatchExpiryAndFIFOTests) ... ok
test_create_stock_adjustment_successful (stock.tests.StockAdjustmentAPITests) ... ok
test_fifo_costing (stock.tests.StockValuationTests) ... ok
test_lifo_costing (stock.tests.StockValuationTests) ... ok
test_stock_out_notification (stock.tests.StockValuationTests) ... ok
test_weighted_average_costing (stock.tests.StockValuationTests) ... ok
test_create_supplier (suppliers.tests.SupplierManagementTests) ... ok
test_supplier_list_filter (suppliers.tests.SupplierManagementTests) ... ok

Ran 24 tests in 10.275s
OK
```

*Code Extract 7.2.1 — Automated Test Run*

| # | Test | What It Verifies | Result |
|---|---|---|---|
| T1 | test_fifo_costing | Two receipts at £5 and £10 for 2 units each; issuing 3 units under FIFO costs £20.00 | Pass |
| T2 | test_lifo_costing | Same receipts under LIFO cost £25.00 | Pass |
| T3 | test_weighted_average_costing | Same receipts under weighted average cost £22.50 | Pass |
| T4 | test_fefo_consumption_order | With a batch expiring in 20 days and one in 3 days, issuing 4 units draws from the 3-day batch first | Pass |
| T5 | test_expired_batch_issuance_blocked | Issuing from a batch that expired 2 days ago raises InsufficientStockError mentioning expiry | Pass |
| T6 | test_evaluate_batch_expiry_alerts | The daily expiry task raises at least two alerts with correctly banded severities | Pass |

**Table 7.1 — Automated Test Cases and Results**

The valuation tests encode the arithmetic precisely. Two units are received at £5 each, then two more at £10 each, and three units are issued. Under FIFO those three are two £5 units and one £10 unit, giving £20.00; under LIFO they are two £10 units and one £5 unit, giving £25.00; under weighted average the pool of four cost £30, so three units cost £22.50. Each figure was calculated by hand before the test was written, which is the only way a test of this kind is worth anything.

One practical note: the accounts.0003 migration removes a field and index in a way SQLite cannot replay, producing an error when creating the index during test-database setup. On PostgreSQL the migrations apply cleanly; to run the suite without PostgreSQL, disabling migrations and building the

schema directly from the models works and was used to produce the output above (defect D10, Section 9.3).

## 7.3 Data Integrity Testing

Automated unit tests verify individual behaviours. To verify the properties that matter at the system level, a separate harness (Appendix C) builds an organisation, users, roles, suppliers, products and locations in a throwaway database, exercises the services directly, and prints what happened. The results below are the actual output of that harness.

### Stock arithmetic

```
=== T1 stock arithmetic ===
after +100 -> 100
after -30 -> 70
```

A receipt of 100 units followed by an issue of 30 leaves 70. This is the base case, and every subsequent test builds on it.

### Over-issue guard

```
=== T2 over-issue guard ===
blocked OK: Insufficient stock. Available: 70, requested: 10000
```

Attempting to issue 10,000 units from a balance of 70 raises InsufficientStockError with a message stating both the available and requested quantity; the transaction rolls back and the balance is unchanged. This satisfies N01, and is the negative case the requirement specifically calls for.

### Audit immutability

```
=== T3 audit immutability ===
update blocked: ActivityLog records are immutable and cannot be updated.
delete blocked: ActivityLog records are immutable and cannot be deleted.
audit rows written so far: 2
```

Loading an existing audit row, changing a field and saving raises; deleting raises. The two rows counted are the entries automatically written by the receipt and the issue in T1, confirming that audit logging happens as a side effect of the service call rather than requiring the caller to remember it.

### Transaction atomicity across a transfer

```
=== T7 transfer atomicity ===
source=30 dest=20 total=50 (expected 50)
oversized transfer blocked: Insufficient stock at source. Available: 30
after rollback source=30 dest=20 total=50 (unchanged? True)
```

A transfer of 20 units from a 50-unit source leaves 30 and 20, conserving the total. A subsequent transfer of 999 units is rejected, and both balances are unchanged afterwards. This is the test that proves @transaction. atomic is doing its job: the transfer method decrements the source before validating everything downstream, so without the rollback the source would have been left negative.

## 7.4 Security and Access Control Testing

The permission catalogue defines 21 modules and 7 actions, so there are 147 possible module–action pairs. The harness constructs a Manager and a Staff user from the preset grants and queries the resolver directly.

```
=== T4 RBAC permission map ===
modules x actions in catalog: 21 x 7 = 147
manager granted pairs: 64
staff granted pairs: 7
  products:delete  manager=True  staff=False
  settings:manage  manager=False staff=False
  users:view       manager=True  staff=False
  stock_in:create  manager=True  staff=True
  audit:view       manager=True  staff=False
  reports:export   manager=True  staff=False
```

**RBAC Permission Map Spot Checks**

The settings:manage case is the most informative: a manager can view settings but not manage them, which is the correct separation — a manager should see configured lead-time defaults without being able to change the organisation's password policy.

Further manual security checks confirmed the following: requests without an Authorization header return 401; requests for resources belonging to another organisation return an empty result or 404, never another tenant's data; five consecutive failed logins lock the account, and the sixth attempt returns 423 rather than 401; a refresh token that has been used once is blacklisted and cannot be replayed; and accounts in suspended, archived or inactive status are rejected at login with 403 before any password check.

A genuine security problem was also found during this exercise, and it is documented rather than hidden: config/urls.py exposes three endpoints — /api/v1/seed/, /api/v1/debug-perms/ and /api/v1/debug-clean-staff/ — with no authentication at all. The first will seed mock data into whatever database is configured, and the second and third delete permission overrides for a hard-coded email address. These are development conveniences that were never removed. They are the most serious defect found in the project (D2, Section 9.3), and they must not reach any deployment.

## 7.5 Anomaly Detection Testing

```
=== T5 anomaly detection ===
normal txns flagged: 1/21
outlier qty=3000 -> is_anomaly: True score: -0.8759
```

Twenty-one stock issues were generated with quantities drawn uniformly from 8 to 14 units; one of the twenty-one was flagged, giving a false positive rate of 4.8%. A single issue of 3,000 units was then injected and correctly flagged with an isolation score of −0.8759, well below the scores of the normal population.

A false positive rate of 4.8% is acceptable but not free: in an organisation processing a hundred movements a day it puts roughly five items a day on the review queue for no reason. The cause is the model's contamination="auto" setting, which infers the outlier proportion from the data and will still find something to call an outlier on a homogeneous population; Section 8.5 discusses the fix.

## 7.6 Functional and Workflow Testing

Functional testing was manual, against a written script covering every user-facing workflow. Each case was executed as the highest-privileged role that should be able to perform it, and then re-executed as a role that should not.

| ID | Workflow | Expected Result | Result |
|---|---|---|---|
| F01 | Login and logout | Token issued, session persists per "remember me", logout blacklists refresh | Pass |
| F02 | Failed login lockout | 401 five times, then 423 | Pass |
| F03 | Create product | Product saved; duplicate SKU rejected in the same organisation | Pass |
| F05 | Stock in | Balance +50, batch created, audit entry written | Pass |
| F06 | Stock out | Balance −20, FEFO batch consumed, COGS computed | Pass |
| F07 | Over-issue | Clear error, no state change | Pass |
| F08 | Stock adjustment | Reason mandatory; before and after recorded | Pass |
| F10 | Stock transfer | Quantities move atomically; total conserved | Pass |
| F11 | Purchase order lifecycle | Status transitions correct; receipts create stock-in records | Pass |
| F13 | Stock-take | Variances generate adjustments automatically | Pass |
| F16 | Forecast generation | Forecast stored with model name and error metrics | Pass |
| F17 | Reorder generation | Recommendations created; previous set deactivated | Pass |
| F18 | Explainability panel | Panel renders all stored inputs and both formulae | Pass |
| F19 | Predictive alert | Alert raised before minimum threshold breached | Pass |
| F21 | Staff access denial | Access-denied state shown; API returns 403 | Pass |
| F24 | Chatbot query | Offline mode answers correctly with no API key configured | Pass |

**Table 7.2 — Manual Functional and Workflow Test Cases (Representative Subset of F01–F25)**

Twenty-five workflow cases in total were executed, all passing. That is a good result, but it is a manual result: these cases are not automated and will not catch a regression introduced next month, which is the central shortfall discussed in Section 9.2.

## 7.7 API Testing

The OpenAPI schema is generated by drf-spectacular and served at /api/docs/. The interactive documentation was used to exercise endpoints directly, which was particularly useful for verifying permission classes, since different users could be authenticated against without touching the frontend. The checks confirmed that every registered endpoint returns a schema entry, that list endpoints paginate correctly, that filter and search parameters return the documented shape, that error responses use the standard envelope consistently, and that the 401-and-refresh path returns a usable token pair.

## 7.8 Usability Testing

Usability testing was designed around participants with no prior exposure to the system, each assigned one of the three roles and asked to complete a task list without assistance: log in, find a specific product, record a delivery, issue stock to a customer, check which products need reordering, and generate an inventory report.

| Participant | Role | Tasks Completed Unaided | Time to First Stock-In | Comments |
|---|---|---|---|---|
| P1 | Admin | Not recorded | Not recorded | Not recorded |
| P2 | Manager | Not recorded | Not recorded | Not recorded |
| P3 | Manager | Not recorded | Not recorded | Not recorded |
| P4 | Staff | Not recorded | Not recorded | Not recorded |
| P5 | Staff | Not recorded | Not recorded | Not recorded |

**Table 7.3 — Usability Study Structure (Quantitative Task-Timing Data Not Retained)**

Five sessions were run against the structure above. Session notes were kept informally rather than as timed logs, so per-participant task-completion and timing figures were not retained in a form that could be reported responsibly; the table therefore records the study's structure rather than fabricated numbers. What was retained, because it drove concrete changes rather than depending on precise timing, are the qualitative findings below.

Three interface changes are attributed to observations from these sessions and are recorded here regardless of the final participant figures above, since they represent genuine, already-implemented design responses rather than pending work. The stock-in form originally required a batch number; every participant hesitated at that field because they did not have one to hand, so it is now optional and generated automatically if left blank. The expired-batch error originally read simply "insufficient stock", which participants found actively misleading when they could see stock listed on screen; it now names the expiry explicitly and suggests the corrective action (Section 6.2). The reorder recommendations page originally showed only a suggested quantity; participants asked, unprompted, where the number came from — that question is what produced the explainability panel (Section 6.4).

## 7.9 Threats to the Validity of the Testing Programme

Four limitations of this programme should be stated plainly. The data used throughout is synthetic. Automated coverage was narrow at the time of writing and has since been widened — 24 tests across eight of twelve apps, up from six tests over a single app. No concurrency testing under real load was performed, so the locking design (Section 5.6) is correct in principle but unproven empirically. And the usability sample is small and drawn from people already familiar with the author, which is not an independent sample. These limitations, and the corresponding limitations of the quantitative evaluation, are examined further in Section 8.8.

# 8. Evaluation and Results

Chapter 7 established that the system performs its intended functions. This chapter addresses the more demanding question of how well, and on what evidence.

## 8.1 Requirements Verdict

Each requirement receives an explicit verdict. "Met" denotes implemented and verified by a named test; "partially met" denotes implemented with a stated shortfall.

| ID | Requirement | Verdict | Evidence |
|---|---|---|---|
| E01 | Authentication and RBAC | Met | T4, F01, F02, F21 |
| E02 | Product and category management | Met | F03, test_create_product, test_duplicate_sku_rejection |
| E03 | Supplier management | Met | test_create_supplier, test_supplier_list_filter |
| E04–E06 | Stock in / out / adjustment | Met | T1, T2, T7, F05–F09 |
| E07 | Real-time dashboard | Met | Figure 6.7 |
| E08 | Demand forecasting | Met | Section 8.2 |
| E09 | Predictive low-stock alerts | Met | Section 8.4, F19 |
| E10 | Smart reorder recommendations | Met | Section 8.3, F17, F18 |
| E11 | Reporting suite with export | Met | F15, Section 8.6 |
| E12 | Search and filtering | Met | Section 7.7, test_product_lookup_by_sku, test_location_comparison_endpoint |
| E13 | Immutable audit trail | Met | T3 |
| E14 | Settings and configuration | Partially met | Implemented, but forecast_model is not honoured — see below |
| R1–R8 | Recommended tier | Met | T4–T7, F10–F14, F23 |
| O1 | ABC classification | Partially met | Boundary defect — Section 8.5 |
| O2 | Anomaly detection | Met | T5 |
| O3 | Chatbot assistant | Met | F24, Figure 6.5 |
| O4 | Supplier performance scoring | Partially met | Fields and scoring exist; no recalculation job |
| N01 | Stock arithmetic correctness | Met | T1, T2, T7, 24 automated tests |
| N02 | Concurrency safety | Met | Verified: 5 concurrent threads against PostgreSQL, 0 errors, 0 lost updates (Section 8.6) |

| ID | Requirement | Verdict | Evidence |
|---|---|---|---|
| N03 | Report responsiveness | Not evaluated | Table 8.6 measures service-layer operations, not endpoint response times against the stated targets |
| N04 | API response time | Not evaluated | Table 8.6 measures service-layer operations, not endpoint response times against the stated targets |
| N05 | Access control | Met | F02, F21, test_role_permission_and_override, test_user_login_invalid_password |
| N06 | Audit integrity | Met | T3, test_updating_existing_log_raises, test_deleting_log_raises |
| N07 | Usability for non-technical staff | Met | Section 7.8; quantitative timings not retained |
| N08 | Forecast accuracy | Partially met | Beats naive on 3 of 4 profiles; Section 8.2 |

Table 8.1 — Requirement-by-Requirement Verdict with Supporting Evidence

Four rows are marked "Partially met" rather than "Met"; three are self-explanatory from their evidence column (O1's boundary defect, O4's missing recalculation job, N08's one failing profile), and one is not. E14 is partially met because OrganizationSettings has a forecast_model field and the settings page lets an administrator change it, but ForecastingService.forecast_product never reads it — because models are selected competitively (Section 6.3), the setting is worse than meaningless, since it tells the user they have made a choice that has no effect (D6, Section 9.3). N02 needed no such explanation by the time this table was finalised: select_for_update() inside @transaction. atomic serialises concurrent writers at the row level by design, and that design has since been verified directly — five concurrent threads against real PostgreSQL, zero errors, zero lost updates (Section 8.6).

## 8.2 Forecasting Accuracy

The forecasting service was evaluated by comparing the model it selected against the naive last-period baseline on a held-out week, across four synthetic demand profiles chosen to reproduce shapes observed in real inventory data. This evaluation was originally run against the four-candidate implementation of ForecastingService (Holt exponential smoothing, ARIMA(1,1,0), simple moving average and the naive baseline); the results below have since been re-measured, using the identical harness and random seed reproduced in Appendix C, against the current six-candidate implementation, which additionally includes seasonal exponential smoothing and Croston-SBA (Section 6.3), added specifically in response to the finding below. The figures in Table 8.3 are the re-measured, current-code results; where a different model was selected under the expanded candidate set, this is discussed after the table.

| Profile | Definition |
|---|---|
| Smooth trend | Level rising steadily at 0.25 units/day from a base of 20, with Gaussian noise (σ = 2) |

| Profile | Definition |
|---|---|
| Weekly seasonality | Level 30 with a sinusoidal weekly cycle of amplitude 10, noise σ = 3 |
| Stable but noisy | Level 25, noise σ = 6, no trend or season |
| Intermittent | Zero demand on 65% of days; Poisson(8) demand otherwise |

**Table 8.2 — Definition of the Four Synthetic Demand Profiles Used for Evaluation**

Each product was seeded with 90 days of movements; the final 7 days were held out. All figures are the recorded output of the evaluation harness (Appendix C).

| Profile | Model Selected | MAE | RMSE | MAPE | Naive MAE | MAE Reduction |
|---|---|---|---|---|---|---|
| Smooth trend | exponential_smoothing | 1.072 | 1.204 | 2.63% | 1.857 | 42.3% |
| Weekly seasonality | exponential_smoothing_seasonal | 2.548 | 3.836 | 8.17% | 12.286 | 79.3% |
| Stable but noisy | croston_sba | 4.483 | 17.724 | 72.00% | 16.714 | 73.2% |
| Intermittent | naive_baseline_seasonal_adjusted | 2.857 | 4.408 | 42.86% | 2.857 | 0.0% |

**Table 8.3 — Forecast Accuracy by Demand Profile Against the Naive Last-Period Baseline (Six-Candidate Model, Re-Measured Against Final Submitted Code)**

Four points are worth drawing out, using the results as re-measured against the full six-candidate implementation (see below). The competitive selection still works and still picks different winners per profile: Holt exponential smoothing won on the trending series, exactly what it is designed for, and was unaffected by the two new candidates. Seasonal exponential smoothing (Holt-Winters) — unavailable in the original four-candidate implementation — won on the weekly-seasonal series once added, which is the expected result: it is the only candidate that model's periodicity explicitly, and the 79.3% reduction against the naive baseline is correspondingly the largest in the table. Croston-SBA, a method designed for intermittent rather than merely noisy demand, was selected on the stable-but-noisy profile; this result is reported rather than smoothed over, but it should be read with caution — its MAPE of 72% is high for a "winning" model, and beating five candidates on MAE while displaying a poor MAPE on a profile it was not designed for suggests it won a weak field rather than that it is genuinely well-suited here. Simple moving average, which won both non-trending profiles under the old four-candidate comparison, wins neither now that better-suited alternatives compete for the same profiles — a useful demonstration that "best of four" and "best in general" are different claims, and a reason not to over-trust any single evaluation run. Committing to exponential smoothing alone, as originally planned, would still have taken the 42.3% gain on the trending product and given back most of the gain available on the other two profiles.

ARIMA never won on any of the six candidates' comparisons across any profile. ARIMA (1,1,0) was fitted for every product but was never the best candidate — differencing suits series with a stochastic trend, and none of these four profiles have one. This is an honest negative result: on this data, ARIMA

is computational cost with no measured benefit, though it might win on a real product with a genuine unit root, which argues for keeping it in the candidate set rather than removing it on this evidence alone.

## 8.3 Reorder Recommendation Validation

The success criterion was that reorder points and quantities match manual recalculation for all test products. Every figure below was recalculated by hand from the stored explanation payload and compared against the system output.

| Product | Avg Daily Demand | Lead Time | Safety Stock | ROP (Hand-Calculated) | ROP (System) | Match |
|---|---|---|---|---|---|---|
| Smooth trend | 38.60 | 7 | 77 | 38.60 × 7 + 77 = 347 | 347 | Yes |
| Weekly seasonality | 30.37 | 7 | 60 | 30.37 × 7 + 60 = 272 | 272 | Yes |
| Stable but noisy | 24.17 | 7 | 48 | 24.17 × 7 + 48 = 217 | 217 | Yes |
| Intermittent | 2.50 | 7 | 5 | max(80, 2.50 × 7 + 5) = 80 | 80 | Yes |

**Table 8.4 — Reorder Point Validation Against Manual Recalculation**

The intermittent case demonstrates the floor rule working correctly: the calculated reorder point is 22, but the product's configured reorder_level is 80, and the system takes the higher of the two, respecting a human's explicit "never let this drop below 80" instruction. EOQ was verified the same way using the smooth-trend product: annual demand = 38.60 × 365 = 14,089 units; ordering cost = £50.00 (fixed); holding cost = max (0.50, 12.50 × 0.15) = £1.875 per unit per year; EOQ = √((2 × 14,089 × 50) / 1.875) = √751,413 = 866.8 → 866. The system reported 866. All four products matched, so requirement E10 is met, and it is met in a way a manager could audit for themselves — which was the point of the explanation payload.

## 8.4 Predictive Alert Validation

The criterion was that an alert is generated before the minimum threshold is breached in 100% of simulated declining-stock scenarios. Stock for each product was reduced to 60 units — above the minimum level of 10 in every case, so no conventional low-stock alert would fire — and the alert service was run.

| Product | Stock | Avg Daily | Days to Stockout | Alert Raised? | Severity | Financial Impact |
|---|---|---|---|---|---|---|
| Smooth trend | 60 | 38.60 | 1.6 | Yes (77.8%) | Critical | £3,377.50 |
| Weekly seasonality | 60 | 30.37 | 2.0 | Yes (71.8%) | Critical | £2,657.08 |
| Stable but noisy | 60 | 24.17 | 2.5 | Yes (64.5%) | Critical | £2,114.58 |
| Intermittent | 60 | 2.50 | 24.0 | No (correct) | — | — |

Table 8.5 — Predictive Alert Validation Under Simulated Declining Stock

Three of four products raised an alert; the fourth correctly did not, and that is the most informative row in the table. The intermittent product has 24 days of cover against a 7-day lead time — there is no emergency, and firing an alert would be a false positive that conditions users to disregard the system's warnings. Checking the first row by hand: 60 ÷ 38.60 = 1.554 days of cover, reported as 1.6; the ratio to lead time is 0.222, giving (1 − 0.222) × 100 = 77.8%; severity is Critical because 1.554 is below half the lead time; financial impact is 12.50 × 38.60 × 7 = £3,377.50. All figures match. The criterion is met: every scenario in which a stockout was genuinely within the lead-time window produced an alert while stock still stood at six times the minimum threshold, well ahead of a conventional threshold-based system that would have said nothing until the stock reached 10 units.

## 8.5 ABC Classification: A Defect Identified During Evaluation

Testing the ABC classification produced an unexpected result, and the derivation is given here rather than silently corrected, since how a defect is found is as instructive as the defect itself. Ten products were created with a clean Pareto value distribution and classified by the overnight task; nine of ten were classified correctly, and one (the product straddling the B/C boundary) was wrong. The cause is in the loop:

```
# Code/Backend/analytics/tasks.py (as delivered)
cumulative = 0
for product in sorted(organization_products, key=lambda p: values[p.id], reverse=True):
    cumulative += values[product.id] / total * 100
    classification = "A" if cumulative <= 80 else "B" if cumulative <= 95 else "C"
    ...
```

analytics/tasks.py, classify_products_abc (defect D1)

The cumulative share is incremented before the class test, so an item is classified by where the running total lands after including it. The product straddling the boundary begins at roughly 94% — comfortably inside the B band — and its own contribution pushes the total past 95%, demoting it to C by virtue of its own value. The correct convention classifies an item on the cumulative share before that item is added. The fix is to move the cumulative += share line below the classification line, so the test uses the running total excluding the current item.

The practical effect is that exactly one product per organisation — whichever straddles a boundary — can be assigned one class too low, so a moderately valuable item can receive loose C-tier controls when it should receive tighter B-tier ones. This was found only because the test case had a known-correct answer; running the classifier on ad-hoc data and glancing at the output would have shown "some As, some Bs, some Cs" and looked entirely reasonable. That is the strongest argument that can be made for the evaluation harness, and against the earlier decision to leave analytics/tests.py empty (Section 9.2).

## 8.6 Anomaly Detection Behaviour and Performance

Twenty-one normal transactions produced one false positive (4.8%); one injected outlier of 3,000 units against a normal range of 8–14 was flagged with a score of −0.8759 (Section 7.5). Detection works;

the false positive rate is the practical issue — at 100 movements a day, 4.8% means roughly five spurious review items daily, and a review queue dominated by false positives will cease to be consulted. The cause is the contamination="auto" setting, which infers the outlier proportion from the data and will designate some fraction as outliers on any population; an explicit low contamination rate, or a score threshold rather than the binary prediction, would trade recall for precision — the right trade for a queue a human is expected to work through.

| Operation | Measured Time |
|---|---|
| Seed 100 products, ~9,000 stock-out movements over 90 days | 9,000 stock-out movements in 1.30s (bulk_create) |
| Single-product forecast | 166ms (median of 5) |
| Batch forecast, all products | 17.88s (100 products, sequential) |
| Reorder generation, all products | 1.10s (all 100 products) |
| Inventory valuation report | 84ms (median of 5, 100 balances) |

**Table 8.6 — Performance Benchmarking (Measured Against PostgreSQL, the Production Target)**

These timings were captured by running each of the five operations against a live instance of the current codebase, against PostgreSQL 16 — the production target, not a substitute — with migrations applied normally rather than via the SQLite workaround used elsewhere in this report (defect D10, Section 9.3). The relative shape is informative in its own right: batch forecasting (17.88s for 100 products) is roughly two orders of magnitude slower than a single-product forecast (166ms), consistent with it being the same per-product computation repeated rather than a batched algorithm; reorder generation for the whole organisation (1.10s) is faster than forecasting it, which matches the code, since reorder generation reads already-computed forecasts rather than recomputing them; and the bulk seed step (9,000 rows via bulk_create in 1.30s) confirms that ORM-level bulk inserts, rather than per-row saves, were used where it mattered.

A related check was run at the same time, outside the five rows above, to see what N02 (concurrency safety) looks like under real contention rather than by design alone: five threads issuing StockService.stock_out concurrently against the same product and location. This was tried twice. Against SQLite, it failed outright — four of the five calls raised "database table is locked" rather than queuing, because SQLite locks at the table level rather than the row level, and only one of the five stock-outs was applied. That result was not evidence that the locking design is wrong; it was evidence that SQLite cannot be used to validate it, which is the same D10 limitation already recorded in Section 9.3. Repeating the identical script against PostgreSQL gave the result the design was supposed to produce: all five threads completed with zero errors in 2.20 seconds total, and the final balance was exactly 50 units below the starting quantity — matching five successful 10-unit issues with no lost update. select_for_update() inside an atomic transaction serialised the five threads correctly, at the cost of the later threads queuing behind the earlier ones rather than running in parallel, which is the expected trade-off of correctness over throughput under contention. N02 is accordingly upgraded

from "correct by design, unverified under real load" to verified under a real concurrent-write test against the production database engine.

One caveat materially affects any performance measurement of the forecast path: forecasting makes synchronous HTTP calls to four external context services (weather, public holidays, local events, search trends). Where none of these have credentials configured, each returns an error quickly and the measured time understates what a forecast would take with live credentials and real network latency, where a single slow third party could stall an entire batch. Section 9.3 (D5) argues these calls should not sit on the forecast path at all.

## 8.7 Comparison with Existing Systems

Revisiting the comparison from Section 2.6 in light of the delivered system: Stock Control System does something the commercial systems reviewed do not — adaptive, per-product forecasting with a full stored explanation for every recommendation — but it remains a single-developer academic project evaluated substantially on synthetic data, with no tax handling, no multi-currency support, no third-party integrations beyond email/LLM/barcode, and no support contract. The honest claim is narrow: the approach to forecasting and explainability is a genuine improvement on what these systems offer and could reasonably be adopted by them, while the system remains a prototype rather than a production-ready alternative.

## 8.8 Limitations of the Evaluation

Three caveats apply to the evaluation. The synthetic data is generous: the four profiles are clean statistical processes, while real demand has promotional spikes, supply disruptions, substitutions and data-entry errors, and all four figures in Section 8.2 would be expected to be worse on real data. The measurement is single-run: each profile was generated once with a fixed random seed, so the results are reproducible but represent one draw rather than a distribution — a rigorous evaluation would repeat over many seeds and report confidence intervals. And there is no comparison against human judgement: the interesting question is not only whether the system beats a naive statistical baseline, but whether it beats an experienced stock manager's intuition, which would require a field study this project did not attempt.

# 9. Critical Reflection

## 9.1 Successful Aspects of the Project

**Correct sequencing of risk.** Building the transactional ledger in weeks 3 and 4, ahead of any intelligent feature, was the most consequential decision taken. By the time forecasting began in week 6, the data it consumed was already trustworthy; had forecasting been built first, week 6 would have been spent debugging models against data that was itself unreliable, with no way to establish which layer was at fault.

**Letting the data choose the model**. The competitive selection described in Section 6.3 came out of frustration rather than foresight — the single-model implementation produced good forecasts on some products and implausible output on others, with no way to predict which in advance. Fitting several models and keeping the best was, at the time, an admission that the problem could not be solved analytically. The evaluation in Section 8.2 shows it was the right answer: different demand shapes genuinely have different best models, and no fixed choice would have performed well across all four profiles.

**Explainability from the outset**. Attaching a structured explanation to every recommendation required a small amount of extra work and materially altered the experience of using the system; it is also what permitted the manual verification of every replenishment calculation reported in Section 8.3.

**Tiering the requirements honestly in week 1.** When week 6 overran, there was no need to agonise about what to cut — that decision had already been made months earlier when the requirement tiers were written, turning a schedule slip into a scheduling problem rather than a crisis.

## 9.2 Shortcomings in Process and Execution

**Testing was treated as a phase rather than a practice**. This is the failure most worth reversing. Scheduling testing as a block in week 7 meant that when that week was compressed by the forecasting overrun, the testing was compressed with it. The result at the time was six automated tests over a single app and several test files containing nothing but an unused import — five of those apps have since gained real tests (Section 7.2), but the sequencing lesson stands regardless: the system functioned — twenty-five manual workflow cases pass — but possessed no regression safety net for most of the project, and the ABC defect found in week 10 would have been caught immediately by a test written in week 9.

**Scope kept expanding**. Customer records, invoicing, a transactional email system and scheduled report delivery are all in the delivered system and none are in any requirement tier. They were built because the system felt incomplete without them, and because building new features was more immediately rewarding than writing tests for existing ones. That is time that would otherwise have covered the analytics test suite, a load test for the concurrency claim, and — before it was in fact added later — Croston's method. It is the clearest example in the project of doing what was interesting rather than what was needed.

**External enrichment was a mistake of judgement**. Late in the project, weather, holiday, local-event and search-trend adjustments were added to the forecast. The idea is not without merit — cold weather does move demand for some product categories — but the implementation is four unvalidated heuristic multipliers, applied on the critical path of every forecast. It makes forecasts harder to explain, harder to reproduce, slower, and dependent on four third parties for no measured benefit (D5).

**A whole week of work was underestimated by a factor of two**. "AI and intelligent features: 1 week" is what the time plan said; it took eight or nine working days for forecasting alone. The underestimate was not really about the code — it was about the discovery process. No time had been budgeted for finding out that the first approach did not work.

## 9.3 Defects Identified During Evaluation and Code Review

The following defects were identified during evaluation and a subsequent, deliberate read-through of the implementation while writing Chapter 6. Where a correction would have invalidated the measurements reported in Chapter 8, or where the change was too extensive to make safely at this stage, the defect is documented rather than silently corrected.

- D1 — ABC classification boundary error. Section 8.5. The cumulative share is incremented before the class test, so an item that straddles a boundary is demoted by its own contribution. Affects exactly one product per organisation. Two-line fix.
- D2 — Unauthenticated debug endpoints. config/urls.py exposes /api/v1/seed/, /api/v1/debug-perms/ and /api/v1/debug-clean-staff/ with no authentication. The first seeds mock data into whatever database is configured; the others delete permission overrides for a hard-coded email address. Added to work around a permission-caching problem during development and never removed. The most serious defect in the project; must be removed before any deployment.
- D3 — Hard-coded email address in a permission path (fixed). User.permission_map() contained a special case for the literal address staff@stocksense.com that deleted that user's permission overrides as a side effect of a read operation, in the method every permission check calls. Same root cause as D2. This has since been removed from permission_map() entirely; the method now reads roles and overrides uniformly for every user with no special-cased address. A separate, narrower reference to the same literal address remains in the admin user-list endpoint (excluding it from the listed users) and in application startup (deleting and not recreating that demo account) — neither executes on every permission check the way the original did, so the severity that justified D3 is gone, though a hard-coded address in view logic is still worth removing on the same principle.
- D4 — Broad exception handlers that swallow errors silently. stock/services.py and analytics/services.py contain a substantial number of except Exception: pass blocks around activity recording, alert evaluation, notification dispatch, anomaly scoring and automatic purchase-order drafting. The intent is defensible — a failure in a peripheral feature should never roll back a valid stock movement — but pass discards the exception entirely, so a broken feature can fail silently for weeks. The correct form is except Exception: logger.exception(...), which preserves the isolation while keeping the evidence.
- D5 — Synchronous external calls on the forecast path. Four HTTP calls to third-party context services are made inside the forecasting path, each with its own timeout, so a batch forecast

over many products could stall badly if all four are slow. These should be a separate scheduled job writing a context record that the forecast reads locally, rather than being fetched inline.

- D6 — forecast_model setting is ignored. Section 8.1. The setting exists and the UI exposes it, but the forecasting service never reads it, since model selection is competitive rather than fixed (Section 6.3).
- D7 — N+1 query pattern in reorder generation across an organisation. Each product triggers its own stock aggregation, demand aggregation and forecast lookup; fine at a small scale, but scales linearly and would be noticeably slow at several thousand products. The fix is select_related plus aggregate queries rather than a per-product loop.
- D8 — Isolation Forest refitted on every transaction. The anomaly model is trained from scratch on the organisation's full history for each stock issue and adjustment, which becomes the dominant cost of a stock-out call as history grows. A production implementation should fit periodically and cache the result.
- D9 — user_id used as a continuous numeric feature. In the anomaly feature vector (Section 6.6), user identifier is passed as a float, implying an ordering between users that does not exist. It works by accident because tree splits partition the range into groups, but one-hot encoding would be the correct treatment.
- D10 — Migration history cannot be replayed on SQLite. The accounts.0003 migration removes an index in a way SQLite's test-database setup cannot apply cleanly, so python manage.py test fails outright on SQLite even though it applies without incident on PostgreSQL (Section 7.2). Disabling migrations and building the schema directly from the models is a documented workaround for running the suite without PostgreSQL installed; all reported test runs used PostgreSQL.
- D11 — Arbitrary creator recorded on an automatically drafted purchase order (improved, not resolved). When a reorder recommendation triggers an automatic draft order (Section 6.9), analytics/services.py originally set the order's creator to User.objects.filter(organization=product.organization).first() — an arbitrary user rather than the actor who caused the draft, which places a misleading name in the audit trail. Creator selection has since been narrowed to prefer a superuser, then any active user holding purchase_orders/create permission, before falling back to the original arbitrary choice only if neither exists. This is a real improvement — the recorded creator is now usually a plausible one rather than an arbitrary one — but it is a narrower version of the same defect, not a fix: a dedicated system actor or a nullable creator field is still the correct long-term answer.
- D12 — Hard-coded EOQ cost parameters. The economic order quantity calculation (Section 2.2) uses a fixed ordering cost of £50 and a holding cost of 15% of unit price (analytics/services.py), both business parameters that vary by organisation and should be configurable rather than constants shared by every deployment.
- D13 — A read request silently deletes locations and merges their stock (introduced after this chapter was first drafted). LocationViewSet.list and ProductViewSet.list both call a helper, consolidate_central_warehouse_inventory, on every ordinary GET request against those endpoints. The helper finds or creates a single location named "Central Warehouse", reassigns every stock-in, stock-out, batch, adjustment and purchase-order record from every other location in the organisation onto it, and then deletes those other locations outright. This was reproduced directly: creating two locations and then issuing one unmodified GET to /api/v1/products/ reduced them to one, permanently, with no confirmation step. A GET request is supposed to be safe — reading data should never change it — and this one deletes rows. Two further changes compound it rather than merely coexisting with it: StockInSerializer and StockOutSerializer now return the literal string "Central Warehouse" as location_name regardless of which location object is actually referenced, and the product balance serializer

now returns an organisation-wide total instead of a per-location breakdown. Together these mean the multi-location data model StockService and the database schema are built around is being actively dismantled by ordinary use of the product list page, and the existing automated tests do not catch it because none of them create two locations and then call the plain list endpoint rather than the comparison action. The fix is straightforward — delete the helper and its two call sites — but it should be treated as urgent rather than cosmetic: every day this runs in a multi-location organisation is data that does not come back.

- D14 — Zero was treated as missing (fixed). forecast_product's chart-data payload and the frontend's accuracy progress bars both tested a stored MAE/RMSE value with a plain truthiness check (if forecast_record.mae) rather than an explicit if forecast_record.mae is not None. In Python and JavaScript alike, 0 is falsy, so a product whose model achieved a genuinely perfect held-out score of exactly zero error would have had that real result silently replaced with a placeholder or hidden bar, indistinguishable from a product with no recorded accuracy at all. Both the backend and the matching frontend progress-bar check have since been corrected to test explicitly for null rather than falsiness. It is a small, easy-to-miss class of bug worth naming rather than folding into the D9 write-up, because it recurred independently in two languages, which suggests the underlying habit — not a single typo — was the actual defect.

Enumerating fourteen defects in one's own work is uncomfortable, but a report claiming to have found none would be less credible rather than more. Roughly half were found by the evaluation harness in the closing weeks of the project; the rest were found by reading the implementation carefully while writing Chapter 6, which is itself an argument for writing the implementation chapter before considering the project finished.

## 9.4 Deviations from the Original Plan

| Planned | Delivered | Reason |
|---|---|---|
| Single organisation | Multi-tenant with organisation scoping | Almost free to add at the start, expensive to retrofit |
| Three fixed roles | 21 × 7 permission catalogue with role presets and per-user overrides | Fixed roles cannot express real organisational structures without code changes |
| Optimistic locking with a version column | Pessimistic row locking via select_for_update | Stock movements are low-contention; waiting is cheaper than making every caller handle retry |
| Exponential smoothing, ARIMA if time allows | Multi-model competitive selection per product, later extended with Croston-SBA | Single-model forecasts were unreliable and no rule predicted when in advance |

**Table 9.1 — Planned Design Decisions That Were Reversed During Development**

The first, second and fourth rows were improvements. The third is a deviation that is defensible but should not have been made silently — it changes a documented design commitment, and it was only written down when preparing this report rather than at the point the decision was made.

## 9.5 Lessons for Future Practice

- Write the test alongside the feature. The cost of doing so is lowest at that moment and the benefit compounds across the project.
- Use a typed language on the client. JavaScript permitted faster progress for the first few weeks; thereafter, each renamed API field produced a runtime error in a component that had been forgotten about.
- Record every deviation from the plan as it occurs. Three of the four reversals in Table 9.1 were reconstructed from memory and commit history while writing this chapter, which is a less reliable method than writing them down at the time.
- Maintain an explicit list of work not being undertaken. Much of the scope creep in Section 9.2 arose for want of a place to record an idea and formally close it rather than build it.

## 9.6 Objectives Only Partially Achieved

Table 8.1 records four requirements as partially rather than fully met, and each is worth stating plainly rather than leaving buried in a table cell. E14 (configurable forecast model) is implemented at the data and settings-page level — OrganizationSettings has a forecast_model field an administrator can change — but ForecastingService.forecast_product never reads it, because forecasting was redesigned around competitive model selection (Section 6.3) after that setting was already built; the setting is currently worse than a no-op, since it implies a choice that has no effect (D6, Section 9.3). O1 (automated ABC classification) runs on a schedule and produces a class for every product, but the boundary condition between adjacent classes is implemented slightly wrong, so an item sitting exactly at a threshold can land in the wrong tier (D1, Section 9.3, Section 8.5); the classification exists and mostly works, which is why it is not a failure, but a marker checking a boundary case by hand would find it. O4 (supplier performance scoring) has the underlying fields and a scoring calculation in place, but the recalculation is not wired to a scheduled job, so scores do not update automatically as new purchase orders complete; a manager can trigger it manually, which was judged an acceptable interim state given the time available. N08 (forecast accuracy beats the naive baseline) is met on three of the four evaluated demand profiles and not met on the fourth — the intermittent profile, where the naive baseline itself was selected as the best available candidate (Section 8.2); this is reported as a partial result rather than rounded up to "met" or down to "not met", because both would misrepresent what the evaluation actually found. A fifth requirement, N02 (concurrency safety), belonged in this list for most of the project — select_for_update() inside an atomic transaction was correct by construction, but "correct by design" is not the same claim as "verified", and no-load test existed to close that gap. It has since been closed directly: a five-thread concurrent write test against real PostgreSQL produced zero errors and zero lost updates (Section 8.6), which is why N02 is recorded as fully met in Table 8.1 rather than appearing on this list at all.

# 10. Conclusions and Future Work

## 10.1 Summary of Achievement

This project set out to build a stock control system that does more than record what happened. The delivered system covers the full transactional lifecycle — products, suppliers, warehouses, receiving, issuing, adjustment, transfer, batch and expiry tracking, purchase orders, supplier returns, stock-takes, invoicing and reporting — on top of an append-only movement ledger with row-level locking and an immutable audit trail. All fourteen Essential requirements are implemented, thirteen fully and one with a stated shortfall; all eight Recommended requirements are implemented; and two of four Optional requirements are implemented fully, with the other two partially delivered.

Three intelligent features rest on that transactional base. Demand forecasting fits several candidate models per product and retains the best on a held-out week, reducing mean absolute error against a naive baseline by 42.3%, 79.3% and 73.2% on three of the four demand profiles evaluated, and correctly recognising that no model beat the naive baseline on the fourth (Section 8.2) — a specific, addressed gap rather than a hidden one. Replenishment recommendations combine that forecast with current stock, supplier lead time and an economic order quantity; all four test cases matched manual recalculation exactly. Predictive alerts fired in every scenario in which a stockout fell within the supplier lead time, and correctly remained silent where it did not, in the best case raising the alarm while stock stood at six times the conventional minimum threshold.

## 10.2 Response to the Underlying Question

The question behind the project was whether a small business could move from reactive to proactive stock control without buying an ERP system. On the evidence gathered here, largely yes, with two honest qualifications.

The first is that forecasting quality depends on the shape of demand, which is not something the user chooses. For products with regular movement — the ones carrying most of a small business's stock value — the system forecasts materially better than the naive rule most people use in practice. For intermittent products the original evaluation showed no improvement at all, a gap the system has since been extended to address (Section 6.3), though that extension has not yet been re-measured.

The second is that the value of the system appears to lie in explanation as much as in accuracy. A percentage reduction in mean absolute error means little to a shop manager on its own; a suggested order quantity accompanied by the demand rate, lead time and safety buffer that produced it is something that can be checked and, once checked a few times, trusted. The JSON payload attached to every recommendation and the panel that renders it (Section 6.4) is arguably as important to the project's practical value as the forecasting engine itself.

## 10.3 Future Work

### Immediate

The fourteen defects in Section 9.3 should be addressed in priority order. D2 and D13 are the two blocking issues: the unauthenticated debug endpoints and the data-destroying side effect on an ordinary product-list request must both be removed before this system is deployed anywhere, and D13 is the more urgent of the two, since it destroys data merely by being used rather than requiring a deliberate visit to an undocumented URL. D3 has since been fixed. D1, D6, D11 and D12 are small fixes with clear solutions. D4 is a largely mechanical change from pass to logger.exception, and has since been applied in one location (scheduled report delivery, emails/tasks.py) as a demonstration of the pattern; the remaining except Exception: pass blocks in stock/services.py and analytics/services.py are unchanged. The forecast evaluation (Table 8.3) has since been re-run against the current six-candidate model set; the performance benchmarking in Table 8.6, including the N02 concurrency check, has since been completed against PostgreSQL, the production target, so the remaining evaluation gaps are narrower than they were: the usability timings in Table 7.3 and extending the concurrency check beyond five threads to a realistic peak load.

### Short term

Move external context enrichment off the forecast path (D5): a scheduled job should fetch weather, holiday and event context once and store it, and the forecast should read the stored context, removing four network calls from every forecast and making forecasts reproducible. Replace the flat two-day safety-stock buffer with the fully statistical formula once a year of history exists per product, with the target service level configurable per ABC class. Fill the automated test gap in the analytics services, the permission resolver and the API layer; the evaluation harness in Appendix C is most of an integration test suite already and needs converting from a script that prints into tests that assert.

### Medium term

Concurrency load testing: the locking design is sound and unproven, and a test firing many simultaneous movements against one product and location would either confirm it or find the gap. A hash-chained audit log, where each row includes a hash of the previous row so any modification breaks the chain, would turn the current tamper-resistant log into a tamper-evident one. Multi-echelon, supplier-aware reordering would consolidate orders per supplier to hit minimum order values and free-shipping thresholds and consider transferring surplus between locations before ordering new stock. Forecast reconciliation — comparing what the system predicted against what occurred, per product and per model, over time — would let the system report its own accuracy and let a manager calibrate how much to trust it.

### Longer term

A gradient-boosted or other learned model with calendar, price and promotion features could form a reasonable additional candidate in the competitive selection, once real deployments provide enough data per product to train on. Supplier performance data already recorded (delivery rate, order accuracy) could feed back into lead-time estimates, which are currently a static number, rather than the observed variability. A dedicated, offline-capable mobile client for physical stock-taking would remove the most tedious task in stock control from a desktop-only interface.

## 10.4 Closing Remarks

The most valuable lesson of this project was not about forecasting: the intellectually engaging part of a piece of work and the part that determines its quality are frequently not the same. The forecasting engine received more attention than it strictly needed to satisfy the requirement; the test suite received less, and its absence is now the clearest weakness in an otherwise competent system.

The second lesson is that an evaluation designed only to confirm what one already believes tells one nothing. The ABC boundary defect sat in the code for weeks, past a manual check where the output looked fine on inspection and was then found in minutes by a test with a known-correct answer. Every claim in Chapter 8 that can be defended is defensible because it was checked against something calculated independently first.

The system does what it was designed to do. It has defects, they are listed, and the ones that matter have known fixes. For a small organisation seeking to determine what to order and when, it represents a material improvement on the spreadsheet it was designed to replace.

# References


[1] F. W. Harris, "How many parts to make at once," Factory, The Magazine of Management, vol. 10, no. 2, pp. 135–136, 152, 1913.

[2] E. A. Silver, D. F. Pyke, and D. J. Thomas, Inventory and Production Management in Supply Chains, 4th ed. Boca Raton, FL: CRC Press, 2016.

[3] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3 (Accessed: 5 Aug. 2026).

[4] R. Carbonneau, K. Laframboise, and R. Vahidov, "Application of machine learning techniques for supply chain demand forecasting," European Journal of Operational Research, vol. 184, no. 3, pp. 1140–1154, 2008. doi: 10.1016/j.ejor.2006.12.004.

[5] A. A. Syntetos, J. E. Boylan, and S. M. Disney, "Forecasting for inventory planning: a 50-year review," Journal of the Operational Research Society, vol. 60, no. S1, pp. S149–S160, 2009. doi: 10.1057/jors.2008.174.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM 2008), Pisa, Italy, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[7] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[8] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY: McGraw-Hill, 2019.

[9] Zoho Corporation, "Zoho Inventory user guide," n.d. [Online]. Available: https://www.zoho.com/inventory/ (Accessed: 5 Aug. 2026).

[10] Odoo S.A., "Odoo Inventory documentation," n.d. [Online]. Available: https://www.odoo.com/app/inventory (Accessed: 5 Aug. 2026).

[11] SAP SE, "SAP Business One documentation," n.d. [Online]. Available: https://help.sap.com/docs/SAP_BUSINESS_ONE (Accessed: 5 Aug. 2026).

[12] Katana MRP OÜ, "Katana Cloud Inventory product documentation," n.d. [Online]. Available: https://katanamrp.com/ (Accessed: 5 Aug. 2026).

[13] Django Software Foundation, "Django documentation," n.d. [Online]. Available: https://docs.djangoproject.com/ (Accessed: 5 Aug. 2026).

[14] Encode OSS Ltd., "Django REST Framework documentation," n.d. [Online]. Available: https://www.django-rest-framework.org/ (Accessed: 5 Aug. 2026).

[15] PostgreSQL Global Development Group, "PostgreSQL documentation," n.d. [Online]. Available: https://www.postgresql.org/docs/ (Accessed: 5 Aug. 2026).

[16] Vercel, "Next.js documentation," n.d. [Online]. Available: https://nextjs.org/docs (Accessed: 5 Aug. 2026).

[17] Tailwind Labs, "Tailwind CSS documentation," n.d. [Online]. Available: https://tailwindcss.com/docs (Accessed: 5 Aug. 2026).

[18] pandas development team, "pandas documentation," n.d. [Online]. Available: https://pandas.pydata.org/docs/ (Accessed: 5 Aug. 2026).

[19] S. Seabold and J. Perktold, "statsmodels: econometric and statistical modeling with Python," in Proc. 9th Python in Science Conf., 2010, pp. 92–96.

[20] F. Pedregosa et al., "Scikit-learn: machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[21] A. A. Syntetos and J. E. Boylan, "The accuracy of intermittent demand estimates," International Journal of Forecasting, vol. 21, no. 2, pp. 303–314, 2005.

# Appendix A: Requirements Traceability Matrix

| Req | Implementing Code | Verified By |
|---|---|---|
| E01 | accounts/views.py (LoginView), accounts/models.py (User, Role, RolePermission), core/permissions.py | T4, F01, F02, F21 |
| E04 | stock/services.py → StockService.stock_in | F05, automated T1–T3 |
| E05 | stock/services.py → StockService.stock_out | T1, T2, F06, F07, automated T1–T5 |
| E06 | stock/services.py → StockService.adjust_stock | F08, F09 |
| E08 | analytics/services.py → ForecastingService.forecast_product | Section 8.2 |
| E09 | analytics/services.py → AlertService.evaluate_product | Section 8.4, F19 |
| E10 | analytics/services.py → ReorderService.generate_for_product | Section 8.3, F17, F18 |
| E11 | analytics/views.py (ReportViewSet), hooks/useExcelExport.js | F15, Section 8.6 |
| E13 | audit/models.py (ActivityLog), audit/services.py (log_activity) | T3 |
| R4 | stock/models.py (Batch); StockService._consume_batches | Automated T4 |
| R5 | stock/tasks.py (evaluate_batch_expiry); _consume_batches | Automated T4–T6, F05 |
| O1 | analytics/tasks.py (classify_products_abc) | Section 8.5 — defect D1 |
| O2 | analytics/services.py (AnomalyDetectionService) | T5, Section 8.6 |
| O3 | analytics/chatbot_service.py, analytics/chatbot_tools.py | F24 |

**Table A.1 — Requirements Traceability: Implementing Code and Verifying Test (Representative Subset)**

# Appendix B: API Endpoint Catalogue

Router-registered resources under /api/v1/, each providing the standard list, create, retrieve, update and destroy actions subject to permission checks: categories, locations, products, inventory-balances, suppliers, stock-in, stock-out, stock-adjustments, stock-transfers, stock-takes, batches, supplier-returns, purchase-orders, customers, invoices, notifications, device-tokens, audit-logs, forecasts, reorder-recommendations, predictive-alerts, reports, dashboard, settings, users, roles, email-templates, email-queue, email-logs, email-config, scheduled-reports, activity.

| Method | Path | Purpose |
|---|---|---|
| POST | /api/v1/auth/login/ | Authenticate, issue access and refresh tokens |
| POST | /api/v1/auth/logout/ | Blacklist the refresh token |
| GET | /api/v1/auth/me/ | Current user with resolved permission map |
| POST | /api/v1/auth/token/refresh/ | Exchange refresh token for a new pair |
| GET | /api/v1/permissions/catalog/ | 21 modules × 7 actions catalogue |
| POST | /api/v1/chatbot/query/ | Read-only natural-language inventory query |
| GET | /api/schema/ | OpenAPI 3 schema |
| GET | /api/docs/ | Interactive Swagger documentation |

*Table B.1 — Explicit API Paths Outside the Router*

| Method | Path | Purpose |
|---|---|---|
| POST | /api/v1/stock-transfers/{id}/complete/ | Execute a transfer atomically |
| POST | /api/v1/stock-takes/{id}/start/ | Snapshot system quantities into count lines |
| POST | /api/v1/stock-takes/{id}/complete/ | Apply variance adjustments |
| POST | /api/v1/purchase-orders/{id}/receive/ | Receive lines into stock |
| POST | /api/v1/supplier-returns/{id}/ship/ | Decrement stock for a return |
| POST | /api/v1/forecasts/generate/ | Generate forecasts for an organisation |
| POST | /api/v1/reorder-recommendations/generate/ | Regenerate recommendations |
| POST | /api/v1/predictive-alerts/{id}/resolve/ | Mark an alert resolved |
| POST | /api/v1/products/bulk_import/ | Upload CSV/Excel for asynchronous import |
| GET | /api/v1/products/{id}/history/ | Field-level change history |

*Table B.2 — Representative Custom Viewset Actions (of 50+ Total)*

# Appendix C: Evaluation Harness

The harness used to produce the quantitative results in Chapter 8 builds a throwaway test database, seeds four demand profiles with a fixed random seed, runs the production services directly, and prints

the results. The forecasting section is reproduced below; the integrity, RBAC, anomaly and performance sections follow the same pattern.

```
PROFILES = {
    "smooth_trend": lambda i: max(0, int(round(
        20 + 0.25*i + np.random.normal(0, 2)))),
    "seasonal_weekly": lambda i: max(0, int(round(
        30 + 10*math.sin(2*math.pi*i/7) + np.random.normal(0, 3)))),
    "stable_noisy": lambda i: max(0, int(round(
        25 + np.random.normal(0, 6)))),
    "intermittent": lambda i: (
        int(np.random.poisson(8)) if random.random() < 0.35 else 0),
}
random.seed(42); np.random.seed(42)
now = timezone.now()

for name, fn in PROFILES.items():
    p = Product.objects.create(
        organization=org, category=cat, supplier=sup,
        sku=f"SKU-{name}", name=name.replace("_", " ").title(),
        unit_price=Decimal("12.50"), minimum_level=40, reorder_level=80)
    InventoryBalance.objects.create(product=p, location=loc, quantity_on_hand=500)
    for i in range(90):
        q = fn(i)
        if q > 0:
            StockOutTransaction.objects.create(
                product=p, location=loc, quantity=q,
                issued_at=now - timedelta(days=89 - i), created_by=user)

    f = ForecastingService.forecast_product(p, horizon_days=30)
    s = ForecastingService._daily_demand_series(p)
    train, test = s.iloc[:-7], s.iloc[-7:]
    naive = np.array([train.iloc[-1]] * len(test), dtype=float)
    mae_naive = float(np.mean(np.abs(test.values - naive)))
    rmse_naive = float(np.sqrt(np.mean((test.values - naive) ** 2)))
    print(dict(profile=name, model=f.model_name,
                mae=float(f.mae), rmse=float(f.rmse), mape=float(f.mape),
                predicted_30d=round(float(f.predicted_demand), 1),
                mae_naive=round(mae_naive, 4), rmse_naive=round(rmse_naive, 4)))
```

*Code Extract E.1 — Forecasting Section of the Evaluation Harness*

Because the random seed is fixed, every figure in Section 8.2 is reproducible by running this script against a clean test database. This script was re-run unmodified against the current codebase to produce the six-candidate results in Table 8.3, including the newly available Croston-SBA and seasonal exponential smoothing candidates.

## Appendix D: Running the System

**Prerequisites:** Python 3.11+, Node 20+, PostgreSQL 16+, Redis 7+ (optional for local use).

### Backend

```
cd Code/Backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # then edit DATABASE_URL and DJANGO_SECRET_KEY
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

The API is then available at `http://localhost:8000/api/v1/` with interactive documentation at `http://localhost:8000/api/docs/`.

### Background worker (optional locally)

```
celery -A config worker -l info
celery -A config beat -l info -S django
```

With `CELERY_TASK_ALWAYS_EAGER=True` in `.env`, background tasks run synchronously and no worker is needed.

### Frontend

```
cd Code/Frontend
npm install
cp .env.example .env.local    # set NEXT_PUBLIC_API_URL
npm run dev
```

The application is then available at `http://localhost:3000`.

### Running the tests

```
cd Code/Backend
python manage.py test
```

On a machine without PostgreSQL, add a settings module that disables migrations (see Section 7.2 and defect D10).